



# WHERE Walk the Future 2026

*International Euregio Summer School on Legged Robots  
from Basics to Frontier Research*

## State Estimation for Legged Robots

Marco Camurri - University of Trento



UNIVERSITÀ  
DI TRENTO

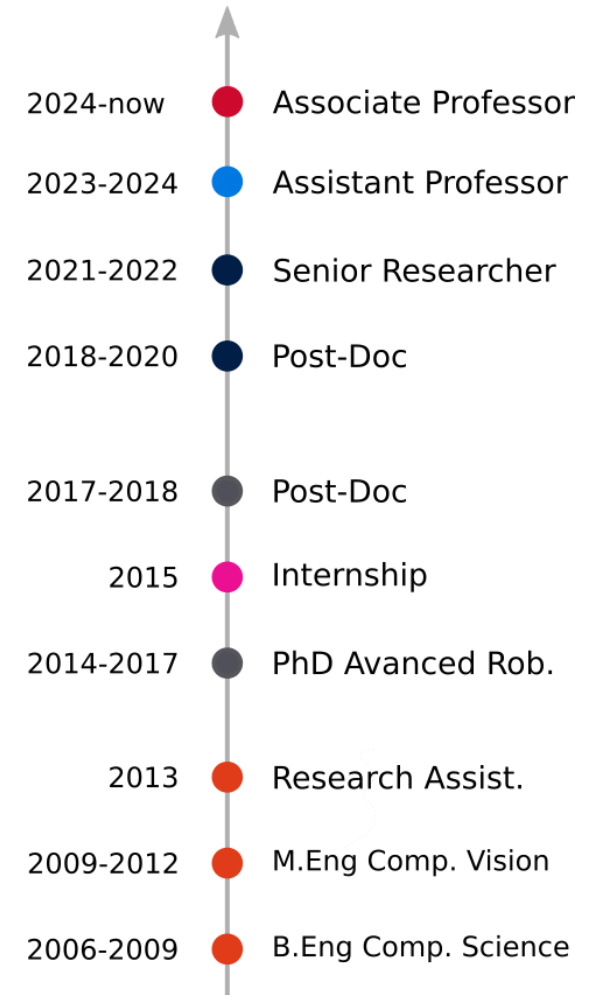
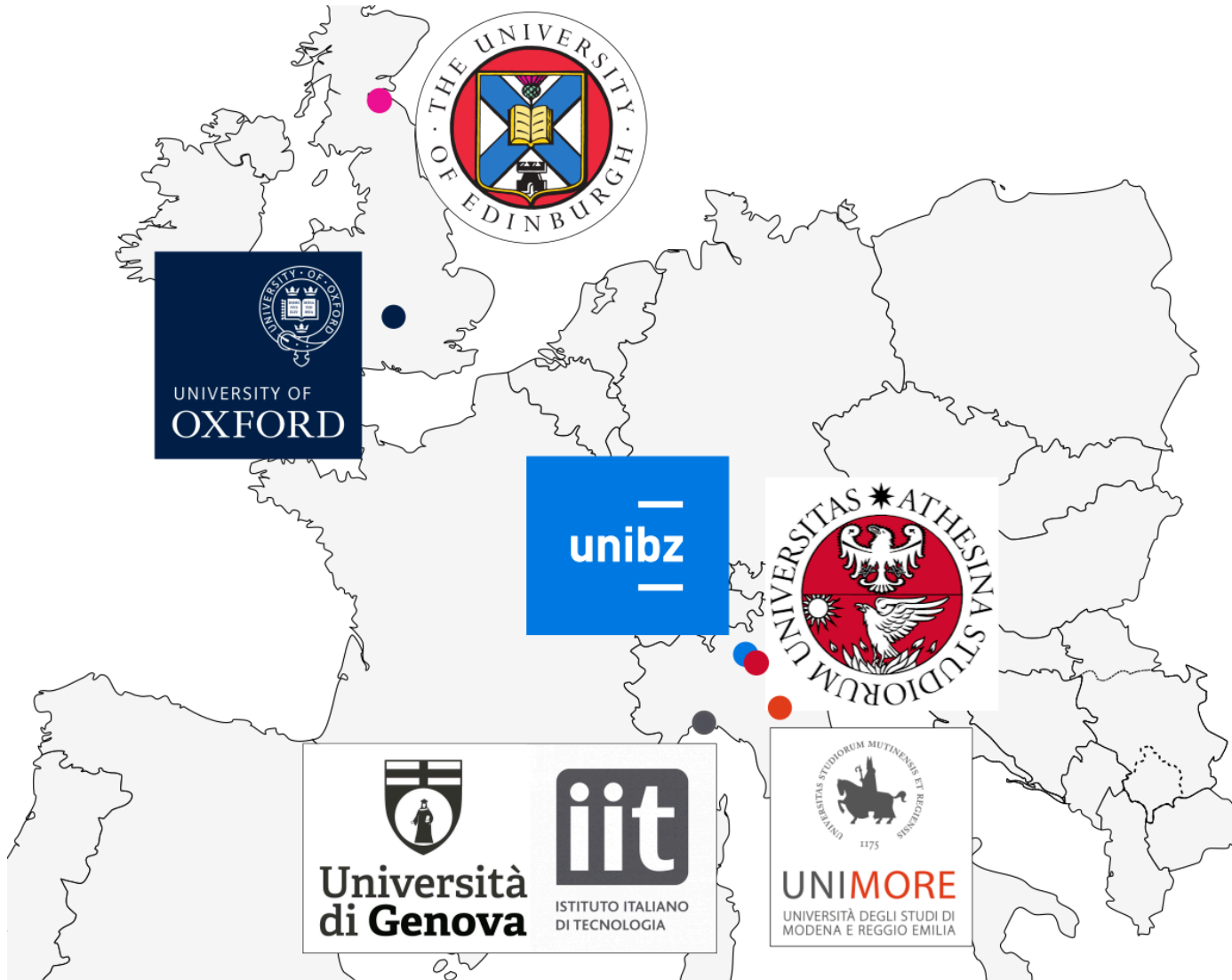


Freie Universität Bozen  
Libera Università di Bolzano  
Università Lìedia de Bulsan



Funded by:

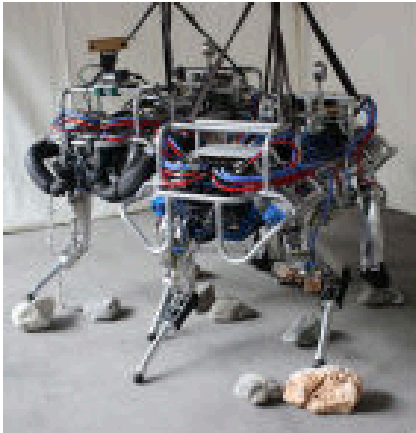
# About Me



# Lots of field trials...

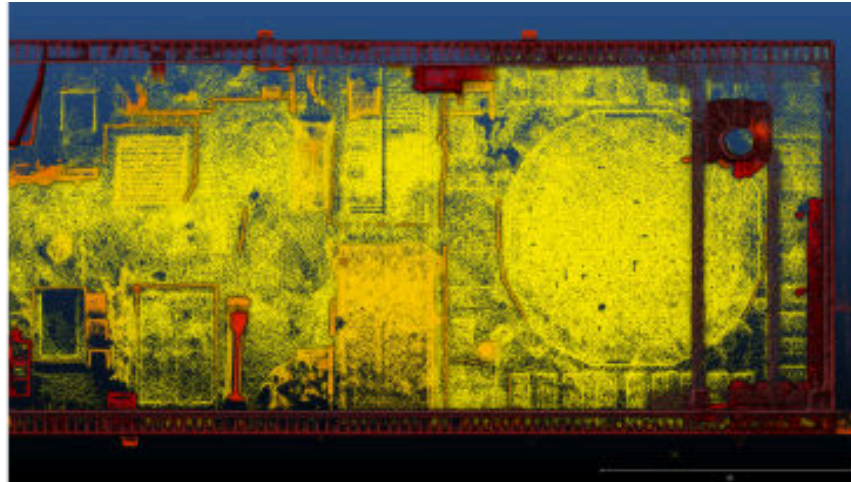


# ... with really cool robots!

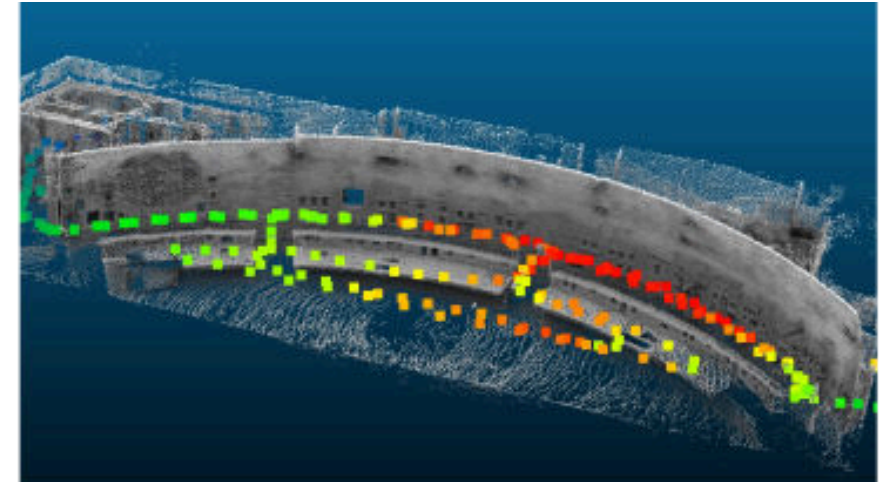


# Mapping the Chernobyl Nuclear Power Plant

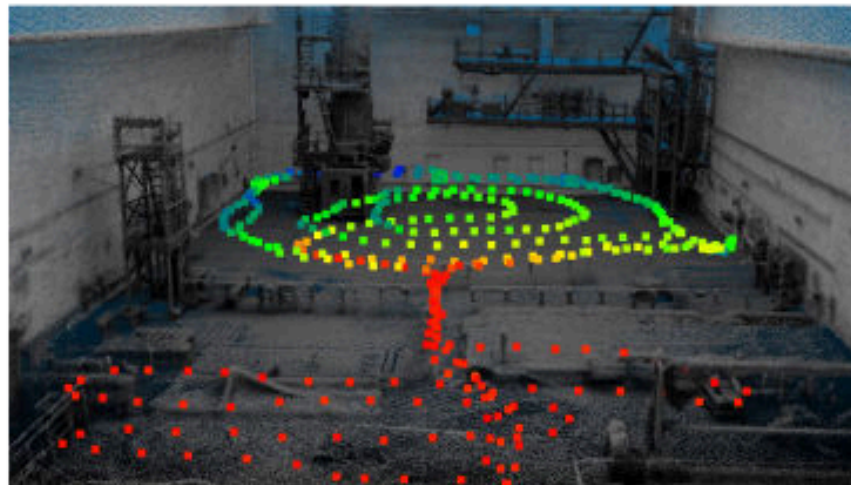
- **VILENS** as motion prior
- **LIDAR SLAM** for loop closure detection and fusion
- **4 radiation sensors** for radiation mapping



Reactor 3 Hall Top view



Reactor 4 Control Room (Mapping and Radiation Scan)



Reactor 3 Hall (Mapping and Radiation Scan)



Reactor 3 Hall (decommissioned)

# Want to work with SLAM?

*Let's stay in touch!*

[marco.camurri@unitn.it](mailto:marco.camurri@unitn.it)

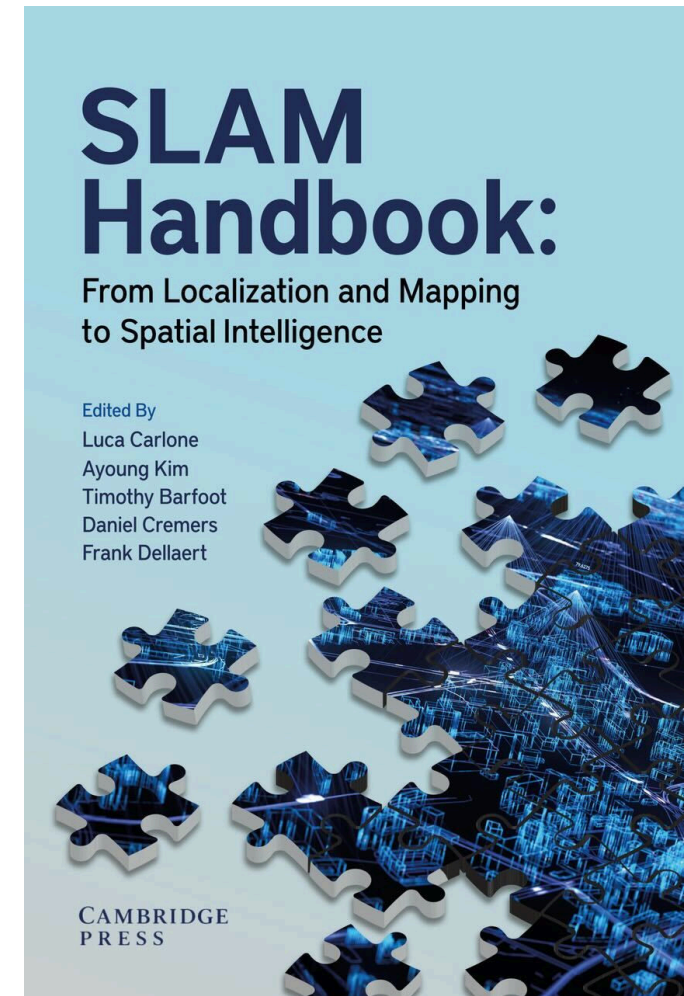


# Schedule

|               | Thursday 24/09                   | Friday 25/09     |
|---------------|----------------------------------|------------------|
| 9:00 - 10:40  | Filtering and Smoothing          | Labs & Tutorials |
| 10:40 - 11:00 | Coffee Break                     | Coffee Break     |
| 11:00 - 12:30 | Inertial-Legged State Estimation | Labs & Tutorials |

# Material for this lecture

- SLAM Handbook (public release draft [here](#))
- *Probabilistic Robotics* D. Fox, S. Thrun, W. Burgard
- These slides, which will be made available
- References (at the end)
- Handwritten notes



# There is no standard notation

|            | Prob robotics | SLAM Handbook            | Barrau         | Hartley                |
|------------|---------------|--------------------------|----------------|------------------------|
| Time step  | $t$           | $k$                      | $n$            | $t$                    |
| State      | $x$           | $\boldsymbol{x}$         | $\chi$         | $\mathbf{X}$           |
| Input      | $u$           | $\boldsymbol{u}$         | $u$            | $\mathbf{u}$           |
| Output     | $z$           | $\boldsymbol{z}$         | $Y$            | $\mathbf{Y}$           |
| Prior      | $\bar{\mu}_t$ | $\check{\boldsymbol{x}}$ | $\chi_{n n-1}$ | $\bar{\mathbf{X}}_t$   |
| Posterior  | $\mu_t$       | $\hat{\boldsymbol{x}}$   | $\chi_{n n}$   | $\bar{\mathbf{X}}_t^+$ |
| Covariance | $\Sigma$      | $\boldsymbol{\Sigma}$    | $P$            | $\mathbf{P}$           |

Be Careful, stay consistent with the source. We won't unify the notation here.

# What is state estimation?

- If you have a control system's background, you know a legged robot can be described as a discrete time-invariant system:

$$\mathbf{x}_k = \mathbf{A}\mathbf{x}_{k-1} + \mathbf{B}\mathbf{u}_k$$

$$\mathbf{y}_k = \mathbf{C}\mathbf{x}_k$$

- If the system is nonlinear, it becomes like this:

$$\mathbf{x}_k = f(\mathbf{x}_{k-1}, \mathbf{u}_k)$$

$$\mathbf{y}_k = h(\mathbf{x}_k)$$

# What is state estimation?

- If the system is noisy, it becomes:

$$\mathbf{x}_k = \mathbf{A}\mathbf{x}_{k-1} + \mathbf{B}\mathbf{u}_k + \mathbf{w}_k$$

$$\mathbf{y}_k = \mathbf{C}\mathbf{x}_k + \mathbf{v}_k$$

- If the system is nonlinear, it becomes like this:

$$\mathbf{x}_k = f(\mathbf{x}_{k-1}, \mathbf{u}_k, \mathbf{w}_k)$$

$$\mathbf{y}_k = h(\mathbf{x}_k, \mathbf{v}_k)$$

- Where  $\mathbf{w}_k$  is the *process noise* and  $\mathbf{v}_k$  is the *measurement noise*
- Both of them are zero-mean Gaussian with covariance  $\mathbf{Q}_k$  and  $\mathbf{R}_k$ :

$$\mathbf{w}_k \sim \mathcal{N}(0, \mathbf{Q}_k) \quad \mathbf{v}_k \sim \mathcal{N}(0, \mathbf{R}_k)$$

# What is state estimation?

From a **control system perspective**, *state estimation* means designing an *observer* that can reconstruct the unknown state  $\mathbf{x}_k$  from the inputs  $\mathbf{u}_k$  and the outputs  $\mathbf{y}_k$

- **What are the inputs?** depending on how the state is defined, they could be actual control actions, or measurements from a sensor
- **What are the outputs?** depending on how the state is defined, they could be direct sensor measurements or pseudomeasurements of part of the state
- **What is the state?** Quantities of interest to control the robot. Typically: absolute orientation and position, linear and angular velocity ...

# What state estimation?

As all systems are noisy, we can also see things from a probabilistic perspective.

In the *probabilistic robotics* literature:

- the system output  $\mathbf{y}_k$  is the measurement (or observation)  $\mathbf{z}_k$
- the objective of *state estimation* is to reconstruct the following probability density function:

$$p(\mathbf{x}_k | \mathbf{z}_{1:k}, \mathbf{u}_{1:k})$$

- assuming all noise to be zero-mean Gaussian, this probability is also Gaussian
- Gaussians are fully defined by their mean and covariance
- state estimation reduces to **estimating mean  $\mu_k$  and covariance  $\Sigma_k$  of  $\mathbf{x}_k$**

**What if we are interested also in estimating the world around the robot?**

Then we move towards the world of Simultaneous Localization and Mapping (SLAM) ...



## What is SLAM ?

*the problem of building a map of the environment, while concurrently estimating its pose with respect to that map*

# What is SLAM

- The ultimate goal of SLAM is to **infer a map representation and robot poses** (i.e., the robot trajectory) **from sensor data**
  - from *proprioceptive sensors* (e.g., wheel odometry or IMU)
  - and *exteroceptive sensors* (e.g., cameras, LiDAR, radars)
- It is an *inverse problem* i.e., given the outcomes of measurements it reconstructs what originated them



# A brief history of SLAM

- Creating maps of the world from observations and measurements is among the oldest challenges in history
- In robotics, the problem can be traced back to the late 80s with Durrant-Whyte
- Definition in 1995 by Durrant-Whyte

# Direct vs. indirect methods

There exist two alternative strategies to solve the SLAM problem:

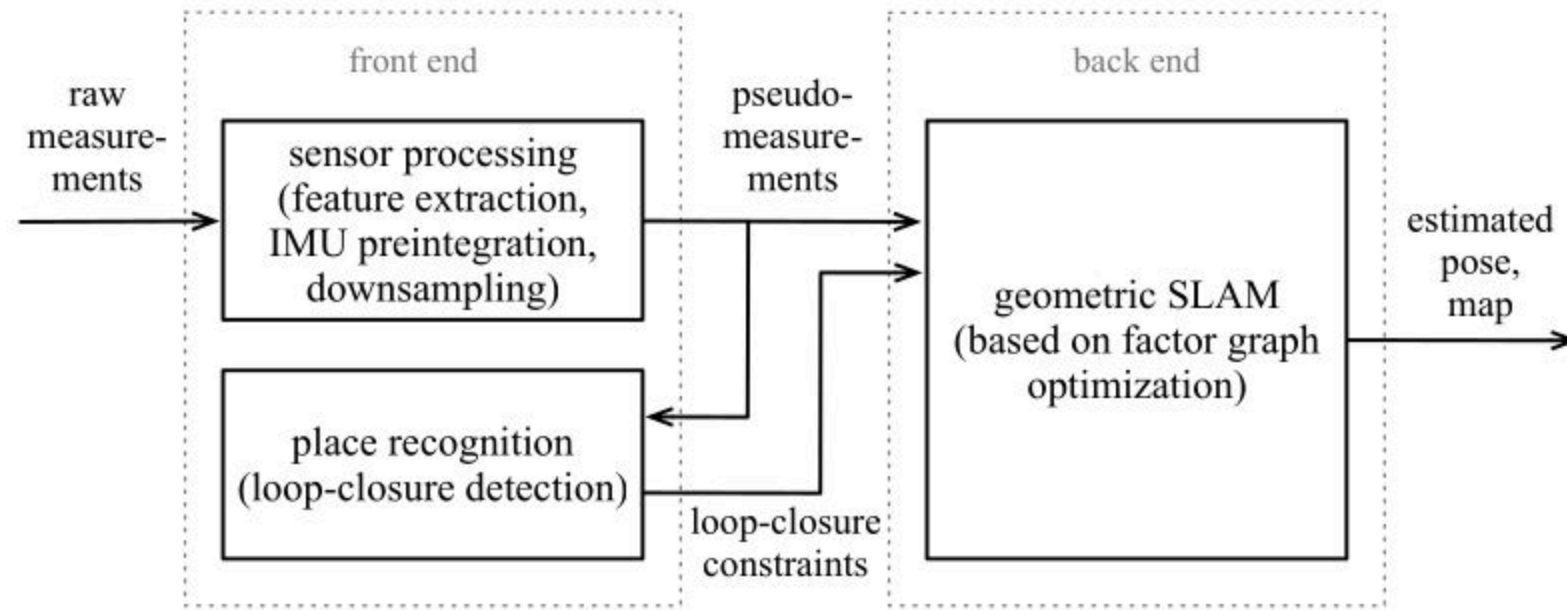
- *Indirect methods* pre-process raw sensor data to extract easier to handle intermediate representations. **Faster, most used**
- *Direct methods* use directly raw sensor data. **Slower, potentially more precise.**

# Terminology

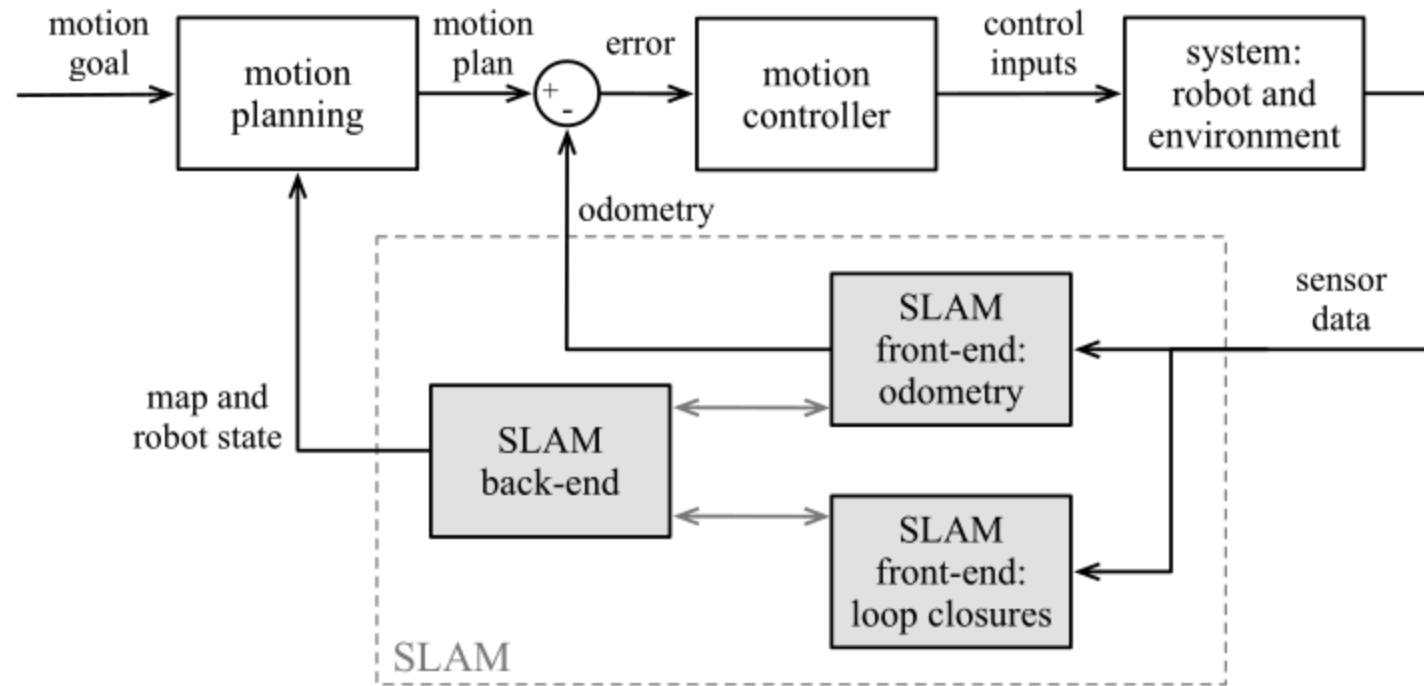
- **Localization** is SLAM with known maps
- **Mapping** is SLAM with known poses
- **Odometry** is SLAM without loop closures
- **State estimation** is basically odometry with velocities and other stuff
- **Bundle Adjustment (BA)** is SLAM without motion model
- **Pose Graph Optimization** is SLAM without landmarks

The line between the above is sometimes blurred...

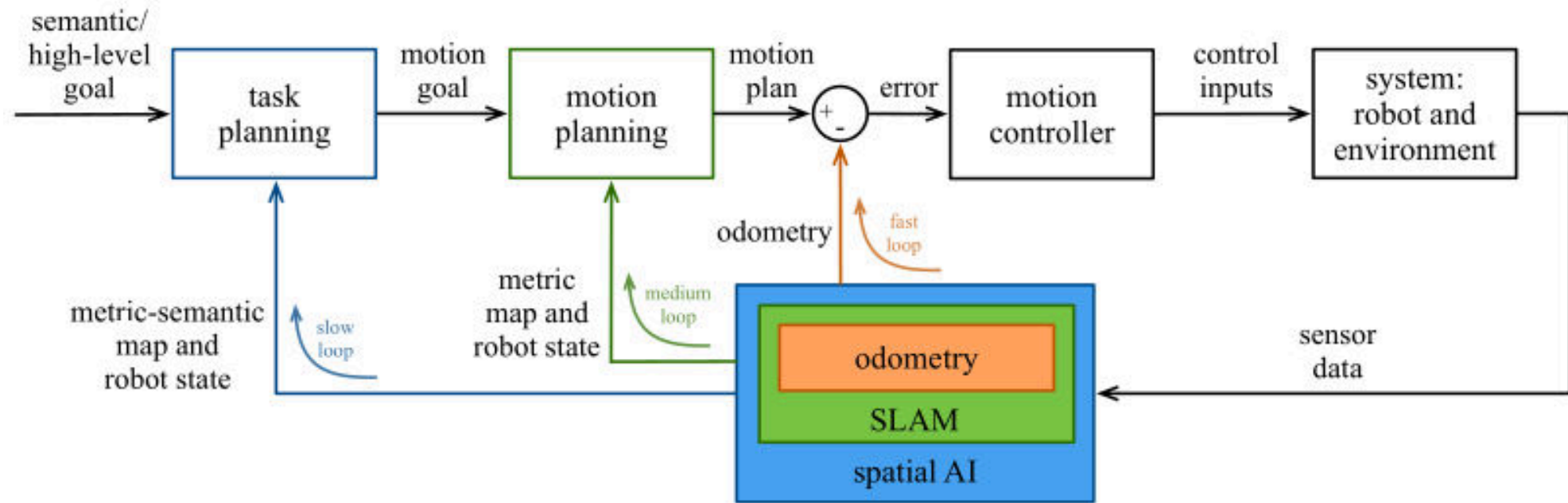
# Anatomy of a SLAM system: indirect methods



# Anatomy of a SLAM system: full system



# The future of SLAM?



# How to build a state estimator?

There are basically two main techniques:

- *Filtering*: estimate the state at the current step, typically with Kalman filters
- *Smoothing*: estimate a history of states, typically with factor graphs

**We will explore both, starting with filtering**

# Bayes Filter

## Definition of Belief

We define the belief at time  $t$  as follows:

$$\mathit{bel}(x_t) = p(x_t \mid z_{1:t}, u_{1:t})$$

## Bayes' Rule

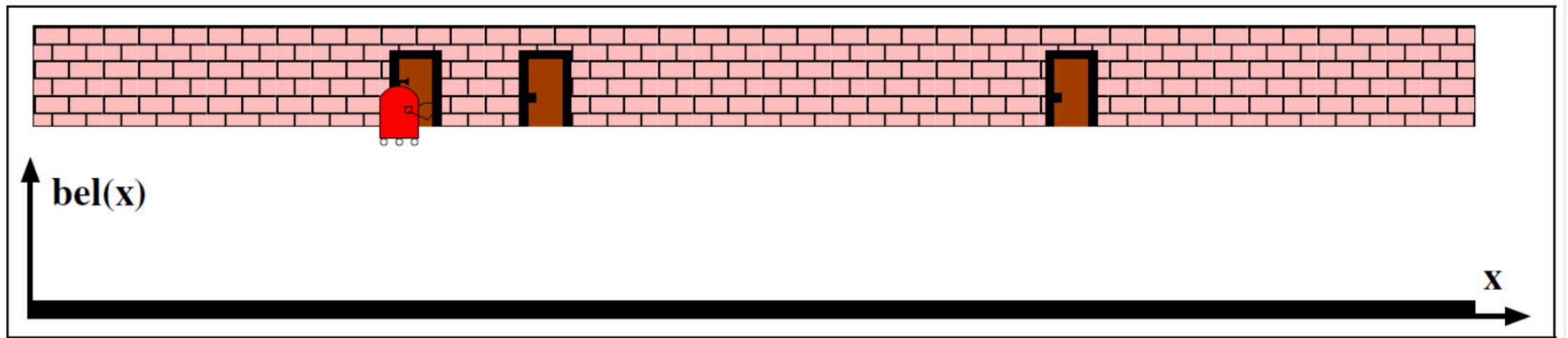
$$p(x|z) = \frac{p(z|x)p(x)}{p(z)}$$

- $p(z)$  is constant, it gives the probability of a measurement regardless of  $x$
- $p(x)$  is the prior
- $p(z|x)$  is the measurement *likelihood*
- $p(x|z)$  is the *posterior* and belief

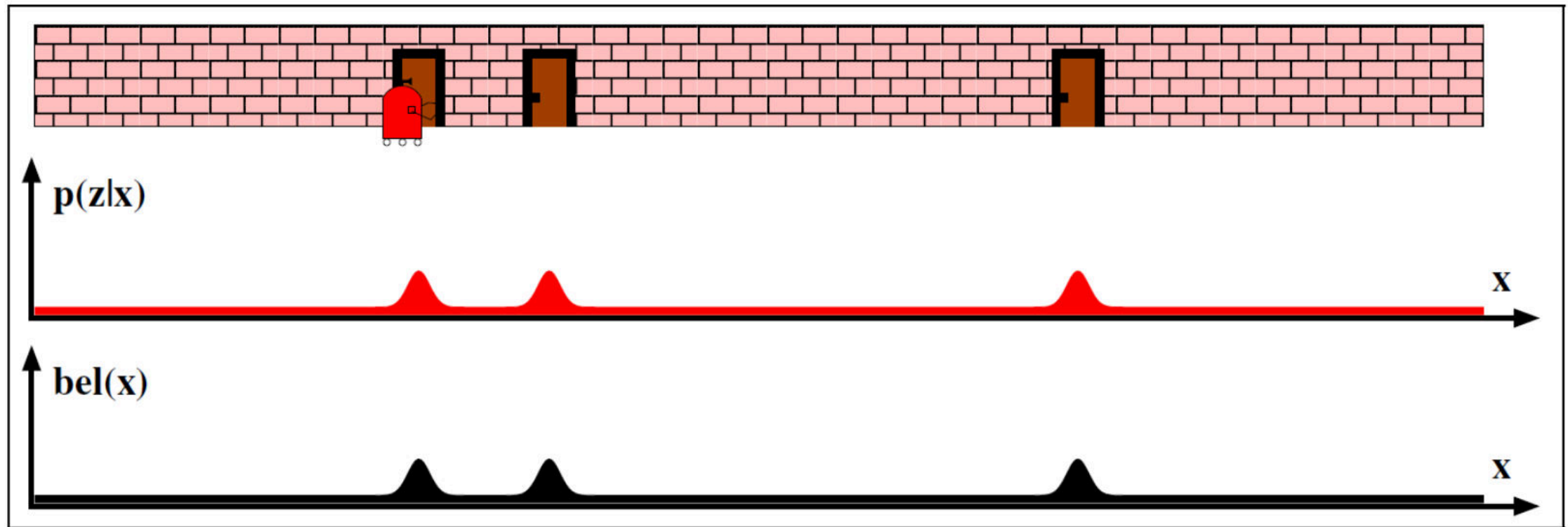
# Recursive Bayes Filter

Let's consider a robot that moves horizontally and can only detect doors

- The state  $x$  is the position along the corridor
- The measurement  $z$  is the binary information "door detected here"
- The control  $u$  is the commanded motion along the corridor

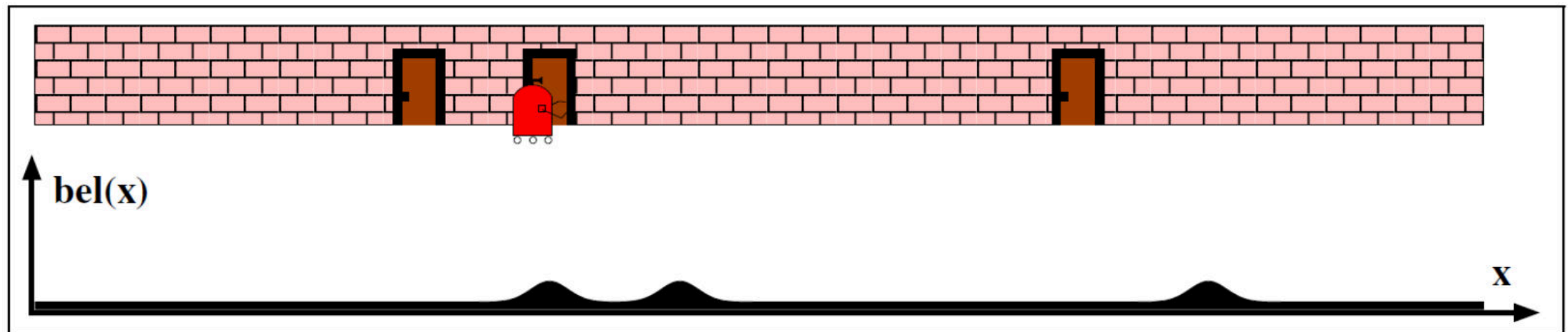


- Given  $x$ , the measurement  $z$  would likely to be like the red distribution  $p(z|x)$
- The prior is uniform, so the posterior is a scaled version of  $p(z|x)$



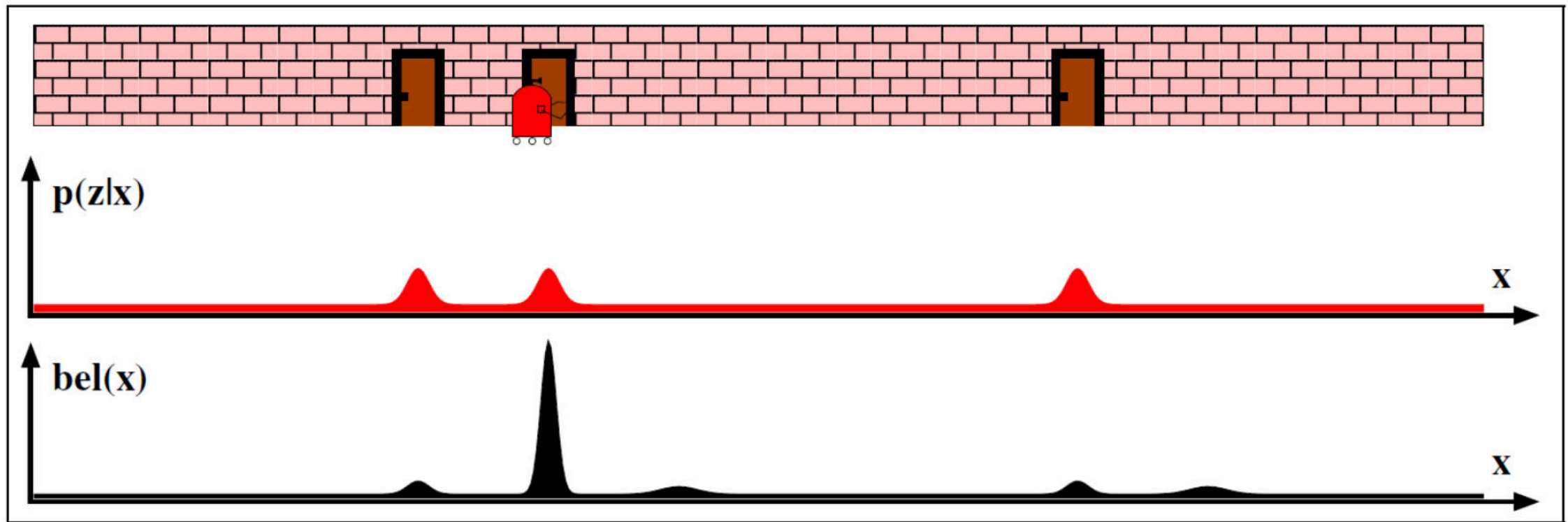
*Courtesy: Thrun, Burgard and Fox*

- Then, the robot is commanded to move. The belief gets propagated
- Note the peaks gets spread across because the robot won't move exactly as it was commanded, there will be some uncertainty related to the error



*Courtesy: Thrun, Burgard and Fox*

- The robot now is in front of another door, it detects it
- The prior already had a peak, the combination of prior and measurement resolves the ambiguity



*Courtesy: Thrun, Burgard and Fox*

# Recursive Bayes Filter: Belief

$$bel(x_t) = \underbrace{p(x_t | z_{1:t}, u_{1:t})}_{\text{definition of belief}}$$

# Recursive Bayes Filter: Bayes' Rule

$$\begin{aligned} bel(x_t) &= p(x_t | z_{1:t}, u_{1:t}) = \\ &= \underbrace{\eta p(z_t | x_t, z_{1:t-1}, u_{1:t}) p(x_t | z_{1:t-1}, u_{1:t})}_{\text{Bayes' rule}} = \end{aligned}$$

# Recursive Bayes Filter: Markov Assumption

$$\begin{aligned} \text{bel}(x_t) &= p(x_t | z_{1:t}, u_{1:t}) = \\ &= \eta p(z_t | x_t, z_{1:t-1}, u_{1:t}) p(x_t | z_{1:t-1}, u_{1:t}) = \\ &= \eta \underbrace{p(z_t | x_t)}_{\text{Markov Assumption}} p(x_t | z_{1:t-1}, u_{1:t}) \end{aligned}$$

# Recursive Bayes Filter: Total Probability

$$\begin{aligned} \text{bel}(x_t) &= p(x_t | z_{1:t}, u_{1:t}) = \\ &= \eta p(z_t | x_t, z_{1:t-1}, u_{1:t}) p(x_t | z_{1:t-1}, u_{1:t}) = \\ &= \eta p(z_t | x_t) p(x_t | z_{1:t-1}, u_{1:t}) = \\ &= \eta p(z_t | x_t) \underbrace{\int p(x_t | x_{t-1}, z_{t-1}, u_{1:t}) p(x_{t-1} | z_{1:t-1}, u_{1:t}) dx_{t-1}}_{\text{Law of total probability}} \end{aligned}$$

# Recursive Bayes Filter: Markov Assumption II

$$\begin{aligned} \text{bel}(x_t) &= p(x_t | z_{1:t}, u_{1:t}) = \\ &= \eta p(z_t | x_t, z_{1:t-1}, u_{1:t}) p(x_t | z_{1:t-1}, u_{1:t}) = \\ &= \eta p(z_t | x_t) p(x_t | z_{1:t-1}, u_{1:t}) = \\ &= \eta p(z_t | x_t) \int p(x_t | x_{t-1}, z_{t-1}, u_{1:t}) p(x_{t-1} | z_{1:t-1}, u_{1:t}) dx_{t-1} \\ &= \eta p(z_t | x_t) \int \underbrace{p(x_t | x_{t-1}, u_t)}_{\text{Markov Assumption}} p(x_{t-1} | z_{1:t-1}, u_{1:t}) dx_{t-1} \end{aligned}$$

# Recursive Bayes Filter: Conditional Independence

$$\begin{aligned} \text{bel}(x_t) &= p(x_t | z_{1:t}, u_{1:t}) = \\ &= \eta p(z_t | x_t, z_{1:t-1}, u_{1:t}) p(x_t | z_{1:t-1}, u_{1:t}) = \\ &= \eta p(z_t | x_t) p(x_t | z_{1:t-1}, u_{1:t}) = \\ &= \eta p(z_t | x_t) \int p(x_t | x_{t-1}, z_{t-1}, u_{1:t}) p(x_{t-1} | z_{1:t-1}, u_{1:t}) dx_{t-1} \\ &= \eta p(z_t | x_t) \int p(x_t | x_{t-1}, u_t) p(x_{t-1} | z_{1:t-1}, u_{1:t}) dx_{t-1} \\ &= \eta p(z_t | x_t) \int p(x_t | x_{t-1}, u_t) p(x_{t-1} | z_{1:t-1}, \underbrace{u_{1:t-1}}_{\text{Independence assumption}}) dx_{t-1} \end{aligned}$$

# Recursive Bayes Filter: Recursive Term

$$\begin{aligned} \text{bel}(x_t) &= p(x_t | z_{1:t}, u_{1:t}) = \\ &= \eta p(z_t | x_t, z_{1:t-1}, u_{1:t}) p(x_t | z_{1:t-1}, u_{1:t}) = \\ &= \eta p(z_t | x_t) p(x_t | z_{1:t-1}, u_{1:t}) = \\ &= \eta p(z_t | x_t) \int p(x_t | x_{t-1}, z_{t-1}, u_{1:t}) p(x_{t-1} | z_{1:t-1}, u_{1:t}) dx_{t-1} \\ &= \eta p(z_t | x_t) \int p(x_t | x_{t-1}, u_t) p(x_{t-1} | z_{1:t-1}, u_{1:t}) dx_{t-1} \\ &= \eta p(z_t | x_t) \int p(x_t | x_{t-1}, u_t) p(x_{t-1} | z_{1:t-1}, u_{1:t-1}) dx_{t-1} \\ &= \eta p(z_t | x_t) \int p(x_t | x_{t-1}, u_t) \underbrace{\text{bel}(x_{t-1})}_{\text{Recursive term}} dx_{t-1} \end{aligned}$$

# Recursive Bayes Filter: Complete Derivation

$$\begin{aligned} \text{bel}(x_t) &= p(x_t | z_{1:t}, u_{1:t}) = \\ &= \eta p(z_t | x_t, z_{1:t-1}, u_{1:t}) p(x_t | z_{1:t-1}, u_{1:t}) = \\ &= \eta p(z_t | x_t) p(x_t | z_{1:t-1}, u_{1:t}) = \\ &= \eta p(z_t | x_t) \int p(x_t | x_{t-1}, z_{t-1}, u_{1:t}) p(x_{t-1} | z_{1:t-1}, u_{1:t}) dx_{t-1} \\ &= \eta p(z_t | x_t) \int p(x_t | x_{t-1}, u_t) p(x_{t-1} | z_{1:t-1}, u_{1:t}) dx_{t-1} \\ &= \eta p(z_t | x_t) \int p(x_t | x_{t-1}, u_t) p(x_{t-1} | z_{1:t-1}, u_{1:t-1}) dx_{t-1} \\ &= \eta p(z_t | x_t) \int p(x_t | x_{t-1}, u_t) \text{bel}(x_{t-1}) dx_{t-1} \end{aligned}$$

# Prediction and Correction Steps

The Bayes filter can be written as a two-step process:

$$bel(x_t) = \underbrace{\eta p(z_t|x_t)}_{\text{Correction}} \underbrace{\int p(x_t|x_{t-1}, u_t) bel(x_{t-1}) dx_{t-1}}_{\text{Prediction}}$$

- Prediction step

$$\overline{bel}(x_t) = \int p(x_t|x_{t-1}, u_t) bel(x_{t-1}) dx_{t-1}$$

- Correction step

$$bel(x_t) = \eta p(z_t|x_t) \overline{bel}(x_t)$$

# Motion and Observation Model

- Prediction step

$$\overline{bel}(x_t) = \int \underbrace{p(x_t | x_{t-1}, u_t)}_{\text{Motion model}} bel(x_{t-1}) dx_{t-1}$$

- Correction step

$$bel(x_t) = \eta \underbrace{p(z_t | x_t)}_{\text{Observation Model}} \overline{bel}(x_t)$$

# Different Realizations

- Bayes filter is a **framework** for recursive state estimation
- There are **different realizations**
- Different properties:
  - Linear vs. non-linear models from motion and observation models
  - Gaussian vs. non-Gaussian distributions
  - Parametric vs. non-parametric filters
  - ...

# Popular Filters

- Kalman filter and EKF
  - Gaussians, linear or linearized models
- Particle filter
  - Non-parametric, arbitrary models (sampling required)

*Apollo 11 (right) used the Kalman filter inside its guidance computer*



## Motion Model

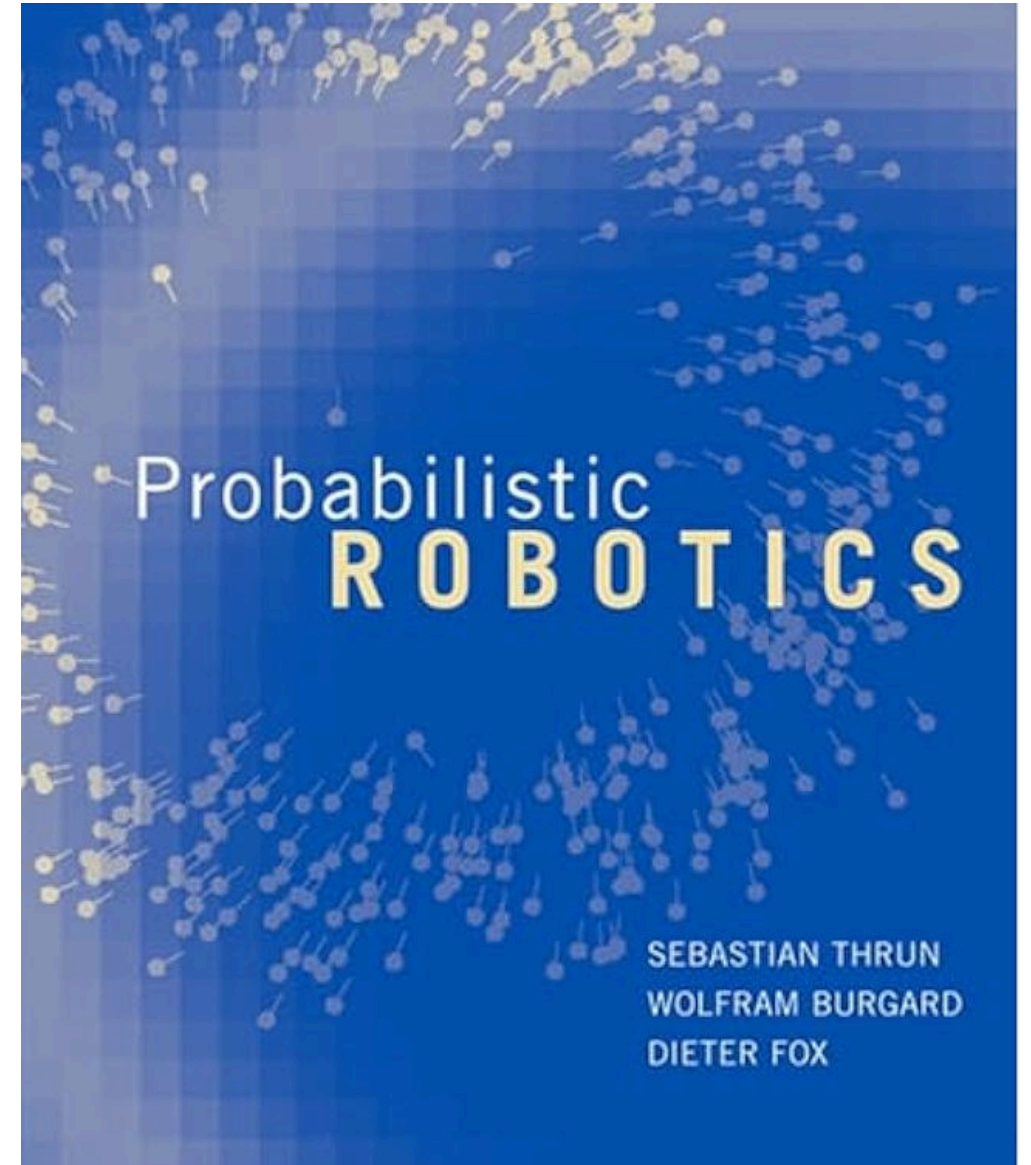
$$\overline{bel}(x_t) = \int \boxed{p(x_t | x_{t-1}, u_t)} bel(x_{t-1}) \, dx_{t-1}$$

## Observation Model

$$bel(x_t) = \eta \boxed{p(z_t|x_t)} \overline{bel}(x_t)$$

# Literature

- Thrun et al. "Probabilistic Robotics" Chapter 2.1 & 2.2 (Probability Primer)
- Thrun et al. "Probabilistic Robotics" Chapter 2.3 (On the Bayes filter)



# Kalman Filter

# History of the Kalman Filter

- Named after Hungarian-born mathematician **Rudolf Emil Kálmán**
- Similar algorithms discovered and developed by T.N. Thiele and P. Swerling
- First described and developed in 1960
- Used on the Apollo 11 guidance computer and countless other aircraft



# Kalman filter in brief

- The Kalman filter is a Bayes filter when the system is *linear* and the distributions are *Gaussian*
- As a Bayes filter:
  - it is a *recursive* algorithm
  - it has two steps: the prediction and the update

# Bayes Filter for State Estimation

Recall the Bayes filter can be written as a two-step process:

$$bel(x_t) = \underbrace{\eta p(z_t|x_t)}_{\text{Correction}} \underbrace{\int p(x_t|x_{t-1}, u_t) bel(x_{t-1}) dx_{t-1}}_{\text{Prediction}}$$

- Prediction step

$$\overline{bel}(x_t) = \int p(x_t|x_{t-1}, u_t) bel(x_{t-1}) dx_{t-1}$$

- Correction step

$$bel(x_t) = \eta p(z_t|x_t) \overline{bel}(x_t)$$

# The Kalman filter is a Bayes filter ...

... where everything is Gaussian

$$p(x) = \det[2\pi\Sigma]^{-\frac{1}{2}} \exp \left( -\frac{1}{2}(x - \mu)^T \Sigma^{-1}(x - \mu) \right)$$

- With *linear* systems and *Gaussian* distributions, the Kalman filter is ***optimal***
- For nonlinear systems, there are tricks to make it work well enough ...

# **How to Update a Gaussian Belief Based on Motions and Observations?**

# Gaussians: Marginalization and Conditioning

- Given

$$x = \begin{bmatrix} x_a \\ x_b \end{bmatrix} \quad p(x) = \mathcal{N}$$

- The marginals are Gaussians ...

$$p(x_a) = \mathcal{N} \quad p(x_b) = \mathcal{N}$$

- ... as well as the conditionals

$$p(x_a|x_b) = \mathcal{N} \quad p(x_b|x_a) = \mathcal{N}$$

# Linear Model for Motions and Observations

# Linear Models

- Both models can be expressed through a linear function

$$f(x) = Ax + b$$

# Linear Models

- Both models can be expressed through a linear function

$$f(x) = Ax + b$$

- A **Gaussian** that is transformed through a linear function **stays Gaussian**

$$p(x) = \mathcal{N} \quad \Rightarrow \quad p(f(x)) = \mathcal{N}$$

# Linear Model

- The Kalman filter assumes a linear transition and observation model
- Zero-mean Gaussian Noise

$$x_t = A_t x_{t-1} + B_t u_t + \epsilon_t$$

$$z_t = C_t x_t + \delta_t$$

# Components of a Kalman Filter

- $A_t$   $n \times n$  matrix describing how the state evolves from time  $t - 1$  to time  $t$  without controls or noise
- $B_t$   $n \times l$  matrix describing how the control  $u_t$  changes the state from time  $t - 1$  to time  $t$
- $C_t$   $k \times n$  matrix describing how to map the state  $x_t$  to an observation  $z_t$
- $\epsilon_t$   $\delta_t$  random variables representing the process and measurement noise that are assumed to be: independent, zero-mean Gaussian distributions with covariance  $R_t$  and  $Q_t$  respectively

# Linear motion model

- Motion under Gaussian noise leads to

$$p(x_t | u_t, x_{t-1}) = ?$$

# Linear motion model

- Motion under Gaussian noise leads to

$$p(x_t | u_t, x_{t-1}) = \det[2\pi R_t]^{-\frac{1}{2}} \exp \left( -\frac{1}{2} (x_t - A_t x_{t-1} - B_t u_t)^T R_t^{-1} (x_t - A_t x_{t-1} - B_t u_t) \right)$$

- $R_t$  fully describes the noise of the motion

# Linear Observation Model

- Measuring under Gaussian noise leads to

$$p(z_t|x_t) = ?$$

# Linear Observation Model

- Measuring under Gaussian noise leads to

$$p(z_t|x_t) = \det[2\pi Q_t]^{-\frac{1}{2}} \exp \left( -\frac{1}{2} (z_t - C_t x_t)^T Q_t^{-1} (z_t - C_t x_t) \right)$$

- $Q_t$  describes the measurement noise

# Does everything remain Gaussian?

- Given an initial Gaussian belief, does the belief stay Gaussian?

$$bel(x_t) = \eta \underbrace{p(z_t|x_t)}_{\text{Gaussian}} \underbrace{\overline{bel(x_t)}}_{?}$$

# Does everything remain Gaussian?

- Given an initial Gaussian belief, does the belief stay Gaussian?

$$bel(x_t) = \eta \underbrace{p(z_t|x_t)}_{\text{Gaussian}} \underbrace{\overline{bel(x_t)}}_{?}$$

- The product of two Gaussians is Gaussian
- We need to prove that  $\overline{bel(x_t)}$  is Gaussian and  $bel(x_t)$  will also be Gaussian

# Does everything remain Gaussian?

$$\overline{bel}(x_t) = \int \underbrace{p(x_t | x_{t-1}, u_t)}_{\text{Gaussian}} \underbrace{bel(x_{t-1})}_{\text{Gaussian}} dx_{t-1}$$

- Is it sufficient to prove  $\overline{bel}(x_t)$  to be Gaussian?

# Does everything remain Gaussian?

$$\overline{bel}(x_t) = \int \underbrace{p(x_t | x_{t-1}, u_t)}_{\text{Gaussian}} \underbrace{bel(x_{t-1})}_{\text{Gaussian}} dx_{t-1}$$

- Would that be sufficient to prove  $\overline{bel}(x_t)$  is Gaussian?
- **Yes!** Convolution of two Gaussians is Gaussian

# Everything stays Gaussian

We can now expand  $\overline{bel}(x_t)$  with the Gaussian definitions

$$\begin{aligned}\overline{bel}(x_t) &= \\ &= \int p(x_t | x_{t-1}, u_t) bel(x_{t-1}) dx_{t-1} \\ &= \eta \int \exp \left( -\frac{1}{2} (x_t - A_t x_{t-1} - B_t u_t)^T R_t^{-1} (x_t - A_t x_{t-1} - B_t u_t) \right) \\ &\quad \exp \left( -\frac{1}{2} (x_{t-1} - \mu_{t-1})^T \Sigma_{t-1}^{-1} (x_{t-1} - \mu_{t-1}) \right) dx_{t-1}\end{aligned}$$

- $\eta$  is a constant collecting the various constants of the Gaussians. NOT the same constant  $\eta$  in the other slides!

# Everything stays Gaussian

Let's define

$$L_t = \frac{1}{2}(x_t - A_t x_{t-1} - B_t u_t)^T R_t^{-1} (x_t - A_t x_{t-1} - B_t u_t) \\ + \frac{1}{2}(x_{t-1} - \mu_{t-1})^T \Sigma_{t-1}^{-1} (x_{t-1} - \mu_{t-1})$$

then

$$\overline{bel}(x_t) = \eta \int \exp(-L_t) \, dx_{t-1}$$

# Everything stays Gaussian

- We can split  $L_t$  in a part that depends on  $x_t$  and on  $x_t, x_{t-1}$

$$L_t = L_t(x_{t-1}, x_t) + L_t(x_t)$$

- Thus we can take  $L_t(x_t)$  out of the integral as it doesn't depend on  $x_{t-1}$

$$\begin{aligned}\overline{bel}(x_t) &= \eta \int \exp(-L_t(x_t) - L_t(x_{t-1}, x_t)) \, dx_{t-1} \\ &= \eta \exp(-L_t(x_t)) \int (-L_t(x_{t-1}, x_t)) \, dx_{t-1}\end{aligned}$$

- Details in *Probabilistic Robotics*, Chapter 3.2 (pp. 46-49)

# Everything Stays Gaussian

- Given an initial Gaussian belief, the belief stays Gaussian

$$\underbrace{\overline{bel}(x_t)}_{\text{Gaussian}} = \underbrace{\int \underbrace{p(x_t|x_{t-1}, u_t)}_{\text{Gaussian}} \underbrace{bel(x_{t-1})}_{\text{Gaussian}} dx_{t-1}}_{\text{Gaussian}}$$

$$\underbrace{bel(x_t)}_{\text{Gaussian}} = \eta \underbrace{p(z_t|x_t)}_{\text{Gaussian}} \underbrace{\overline{bel}(x_t)}_{\text{Gaussian}}$$

Everything is and stays Gaussian 🎉

# How do we represent Gaussians?

- Mean

$$\mu_t = ?$$

- Covariance

$$\Sigma_t = ?$$

- These two pieces of information fully describe the Gaussian distribution
- The objective of the Kalman filter is then to estimate them

# To Derive the Kalman Filter

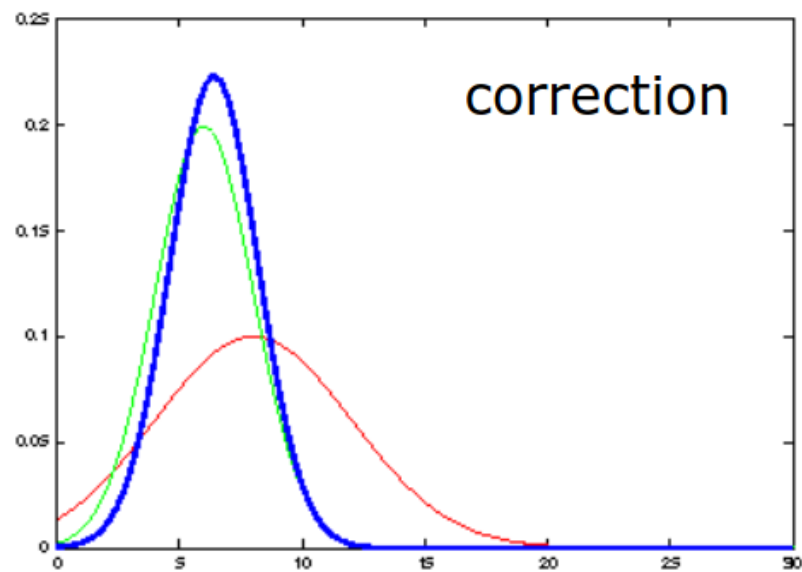
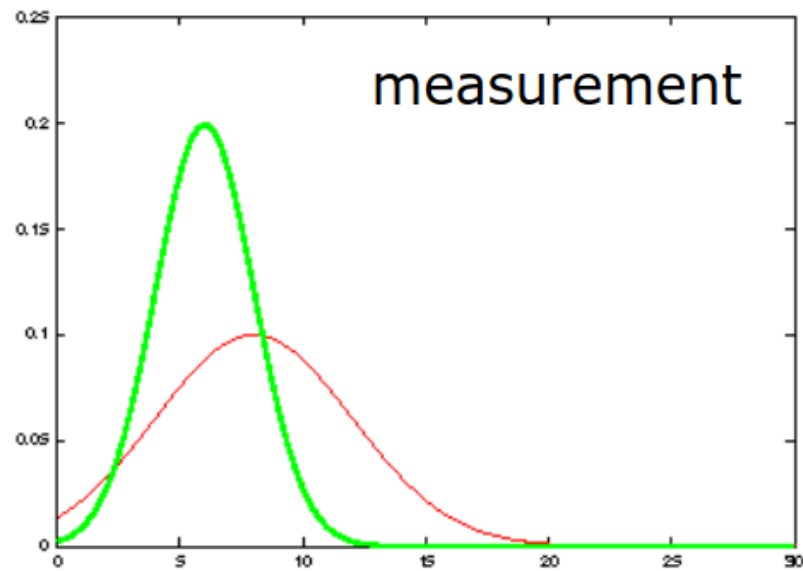
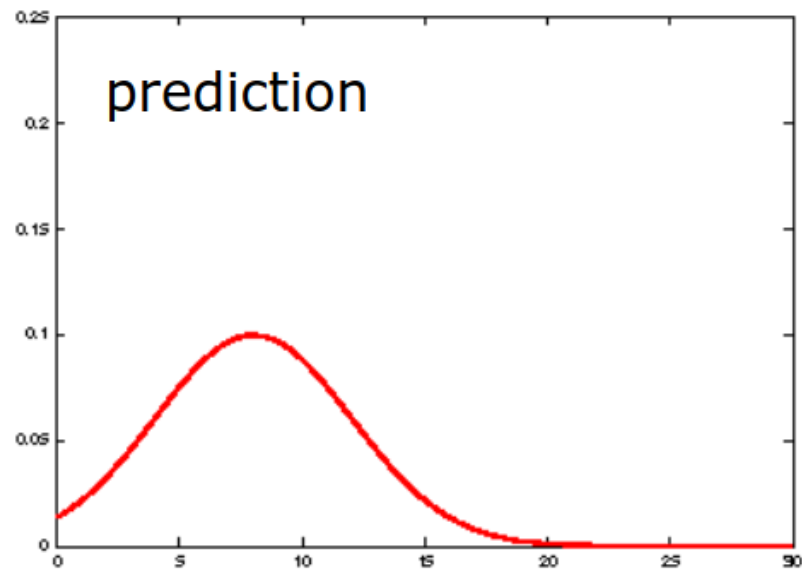
Algorithm, One Exploit...

- Product of two Gaussians is a Gaussian
- Gaussians stays Gaussians under linear transformations
- Marginal and conditional distribution of a Gaussian stays a Gaussian
- Computing mean and covariance of the marginal and conditional of a Gaussian
- Matrix inversion lemma
- ...

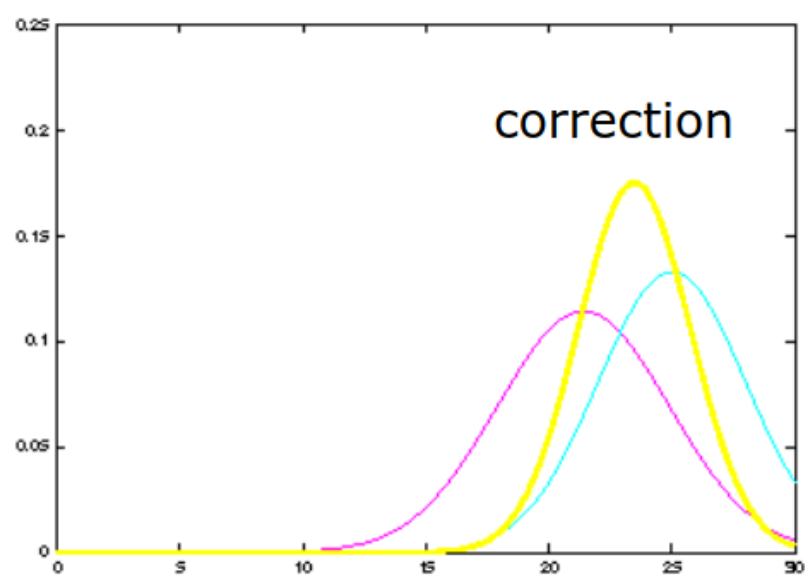
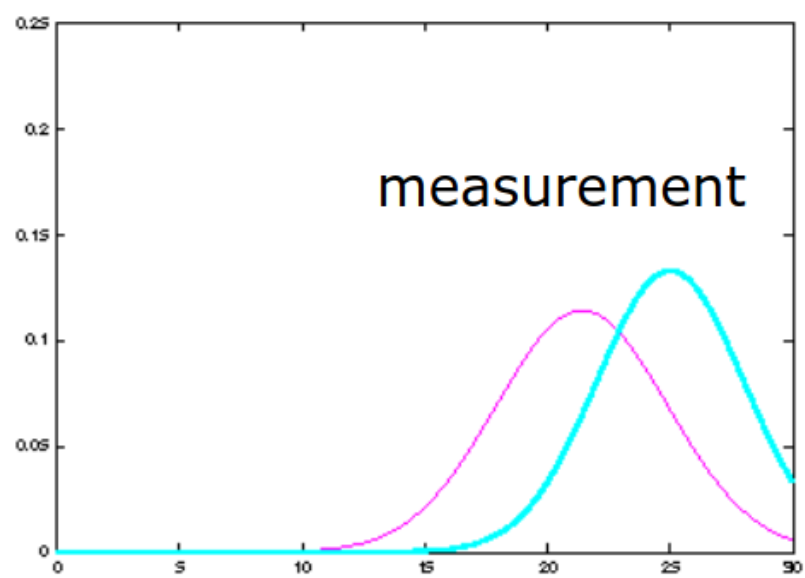
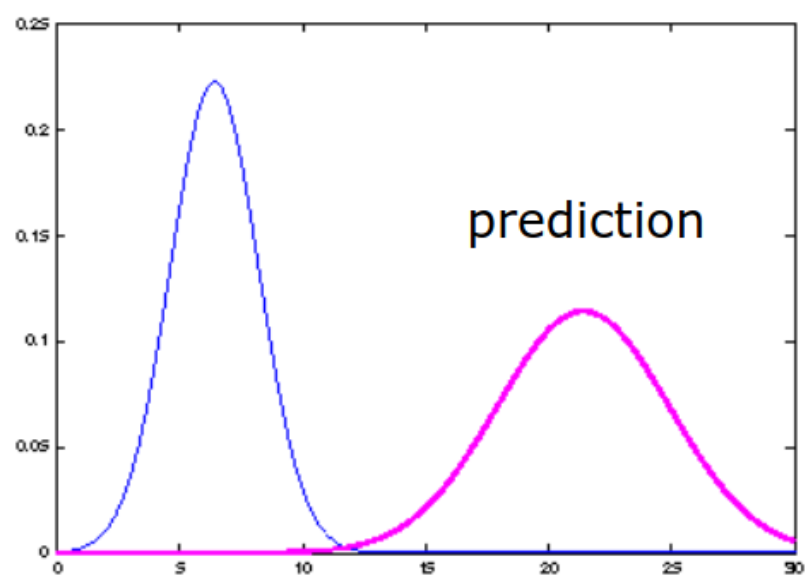
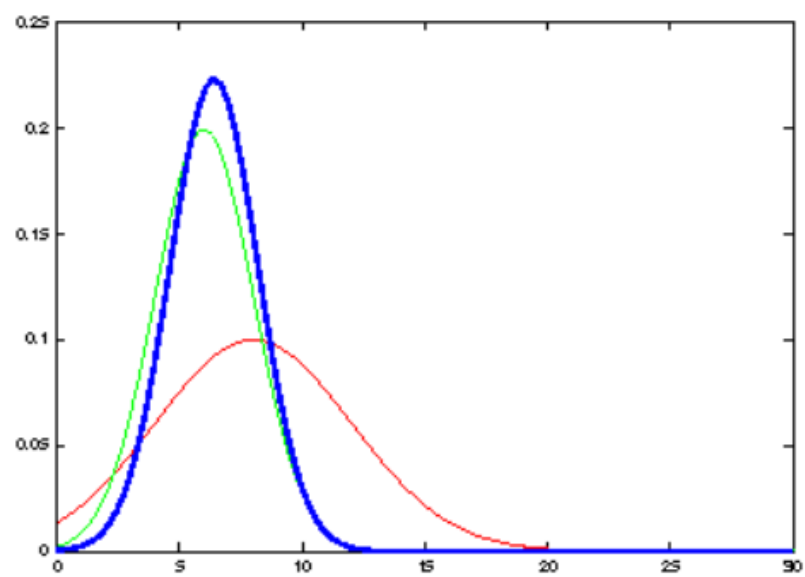
Leading to ...

## ... the Kalman Filter Algorithm

1. Input:  $\mu_{t-1}, \Sigma_{t-1}, u_t, z_t$
2.  $\bar{\mu}_t = A_t \mu_{t-1} + B_t u_t$
3.  $\bar{\Sigma}_t = A_t \Sigma_{t-1} A_t^T + R_t$
4.  $K_t = \bar{\Sigma}_t C_t^T (C_t \bar{\Sigma}_t C_t^T + Q_t)^{-1}$
5.  $\mu_t = \bar{\mu}_t + K_t (z_t - C_t \bar{\mu}_t)$
6.  $\Sigma_t = (I - K_t C_t) \bar{\Sigma}_t$
7. return:  $\mu_t, \Sigma_t$



It's a weighted mean!



# Kalman Filter Algorithm

1. Input:  $\mu_{t-1}, \Sigma_{t-1}, u_t, z_t$
2.  $\bar{\mu}_t = A_t \mu_{t-1} + B_t u_t$
3.  $\bar{\Sigma}_t = A_t \Sigma_{t-1} A_t^T + R_t$
4.  $K_t = \bar{\Sigma}_t C_t^T (C_t \bar{\Sigma}_t C_t^T + Q_t)^{-1}$
5.  $\mu_t = \bar{\mu}_t + K_t (z_t - C_t \bar{\mu}_t)$
6.  $\Sigma_t = (I - K_t C_t) \bar{\Sigma}_t$
7. return:  $\mu_t, \Sigma_t$

What's the meaning of  $K_t$  ?

## A useless sensor ...

- Infinite sensor noise  $Q_t = +\infty$
- Leads to :

$$4. K_t = \bar{\Sigma}_t C_t^T (\cancel{C_t \bar{\Sigma}_t C_t^T} + Q_t)^{-1} \rightarrow 0$$

$$5. \mu_t = \bar{\mu}_t + \cancel{K_t (z_t - C_t \bar{\mu}_t)}$$

- Observation will be ignored

## A perfect sensor ...

- No noise  $Q_t = \text{diag}(0)$
- Leads to:

$$4. K_t = \bar{\Sigma}_t C_t^T (C_t \bar{\Sigma}_t C_t^T + \cancel{Q_t})^{-1} \rightarrow C_t^{-1}$$

$$5. \mu_t = \bar{\mu}_t + C_t^{-1} (z_t - C_t \bar{\mu}_t) = \cancel{\bar{\mu}_t} + C_t^{-1} z_t - \cancel{C_t^{-1} C_t \bar{\mu}_t}$$

$$6. \mu_t = C_t^{-1} z_t$$

- Prediction will be ignored, we just invert the sensor model

# Kalman filter assumptions

- All distributions are Gaussian
- Motion and observation models are linear
- **What if it's not the case?**

# Extended Kalman Filter

# Extended Kalman filter

- All distributions are Gaussian
- Motion and observation models are linear nonlinear

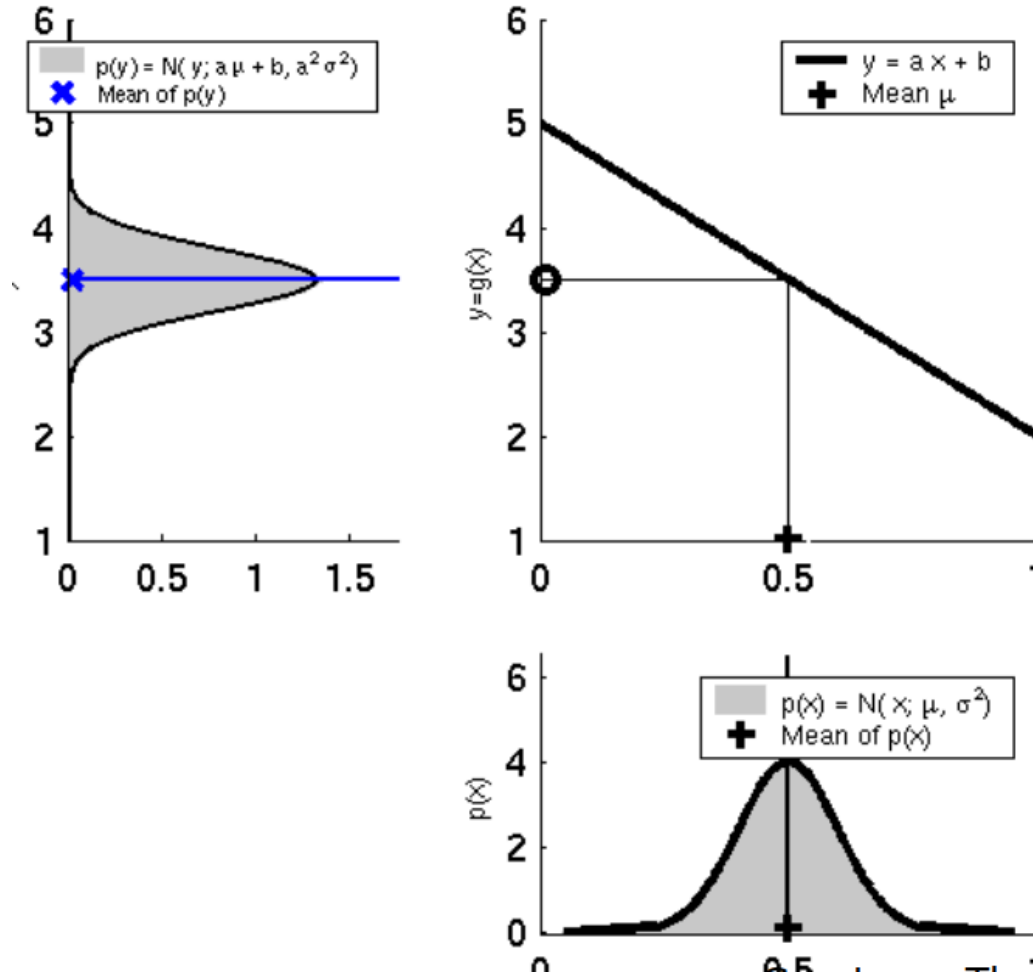
# Nonlinear dynamic systems

- Most realistic problems involve nonlinear functions

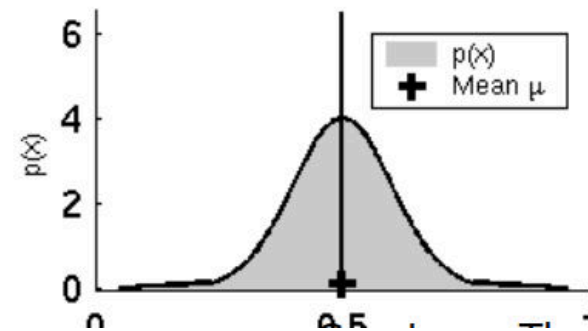
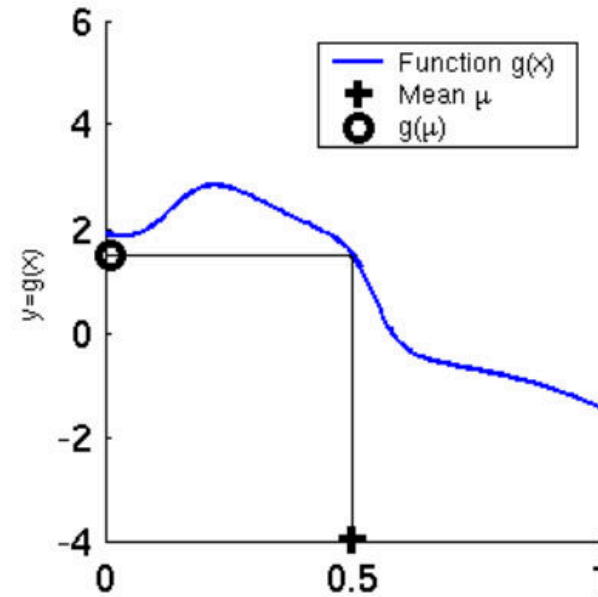
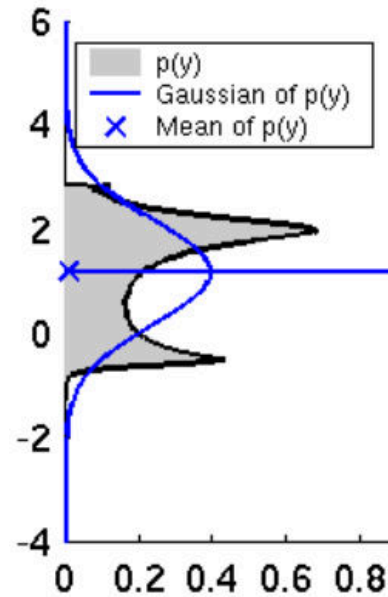
$$x_t = \cancel{A_t x_{t-1} + B_t u_t + \epsilon_t} \rightarrow g(u_t, x_{t-1}) + \epsilon_t$$

$$z_t = \cancel{C_t x_t + \delta_t} \rightarrow h(x_t) + \delta_t$$

# Linearity assumption revisited



# Nonlinear function



# Non-Gaussian distributions

- Nonlinear transformations lead to non-Gaussian distributions!
- The Kalman filter is not usable anymore!

**How do we solve this?**

# Non-Gaussian distributions

- Nonlinear transformations lead to non-Gaussian distributions!
- The Kalman filter is not usable anymore!

## How do we solve this?

💡 Local linearization!

# EKF Linearization: First order Taylor expansion

- Prediction

$$g(u_t, x) \approx g(u_t, \mu_{t-1}) + \underbrace{\frac{\partial g(u_t, x)}{\partial x}}_{G_t} (x - \mu_{t-1})$$

- Correction

$$h(x_t) \approx h(\bar{\mu}_t) + \underbrace{\frac{\partial h(x)}{\partial x}}_{H_t} (x - \bar{\mu}_t)$$

- $G_t$  and  $H_t$  are the Jacobians of  $g(\cdot)$  and  $h(\cdot)$  respect to  $x_t$  computed at  $\mu_{t-1}$  and  $\bar{\mu}_t$  respectively

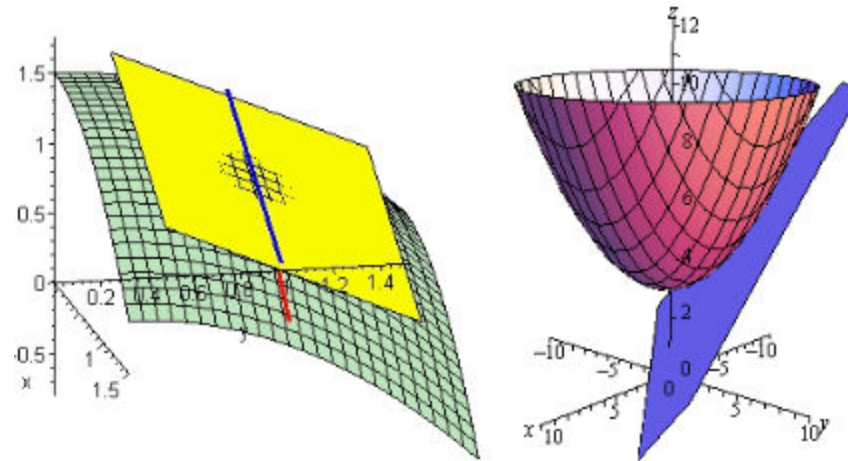
# Reminder: Jacobian

- It is a non-square matrix  $m \times n$  in general
- Given a vector-valued function  $g(\cdot)$ , its Jacobian is defined as:

$$x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \quad g(x) = \begin{bmatrix} g_1(x) \\ g_2(x) \\ \vdots \\ g_m(x) \end{bmatrix} \quad G_x = \begin{bmatrix} \frac{\partial g_1}{\partial x_1} & \frac{\partial g_1}{\partial x_2} & \cdots & \frac{\partial g_1}{\partial x_n} \\ \frac{\partial g_2}{\partial x_1} & \frac{\partial g_2}{\partial x_2} & \cdots & \frac{\partial g_2}{\partial x_n} \\ \vdots & \vdots & \cdots & \vdots \\ \frac{\partial g_m}{\partial x_1} & \frac{\partial g_m}{\partial x_2} & \cdots & \frac{\partial g_m}{\partial x_n} \end{bmatrix}$$

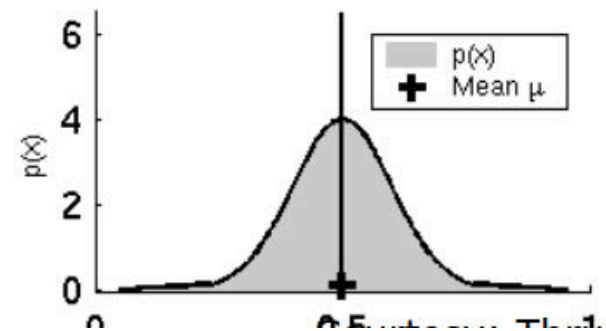
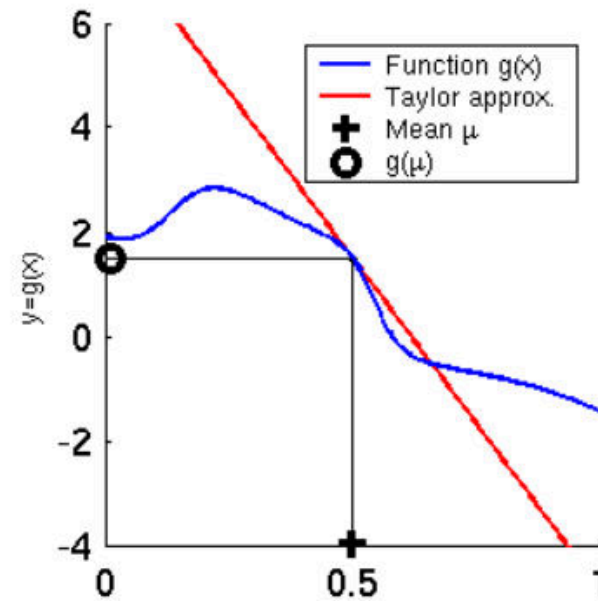
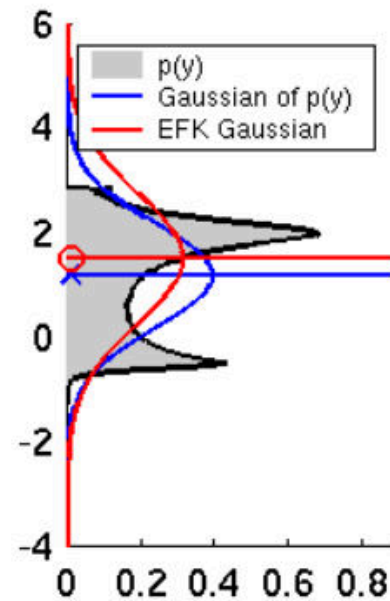
# Reminder: Jacobian

- It is the orientation of the tangent plane to the vector valued function at a given point

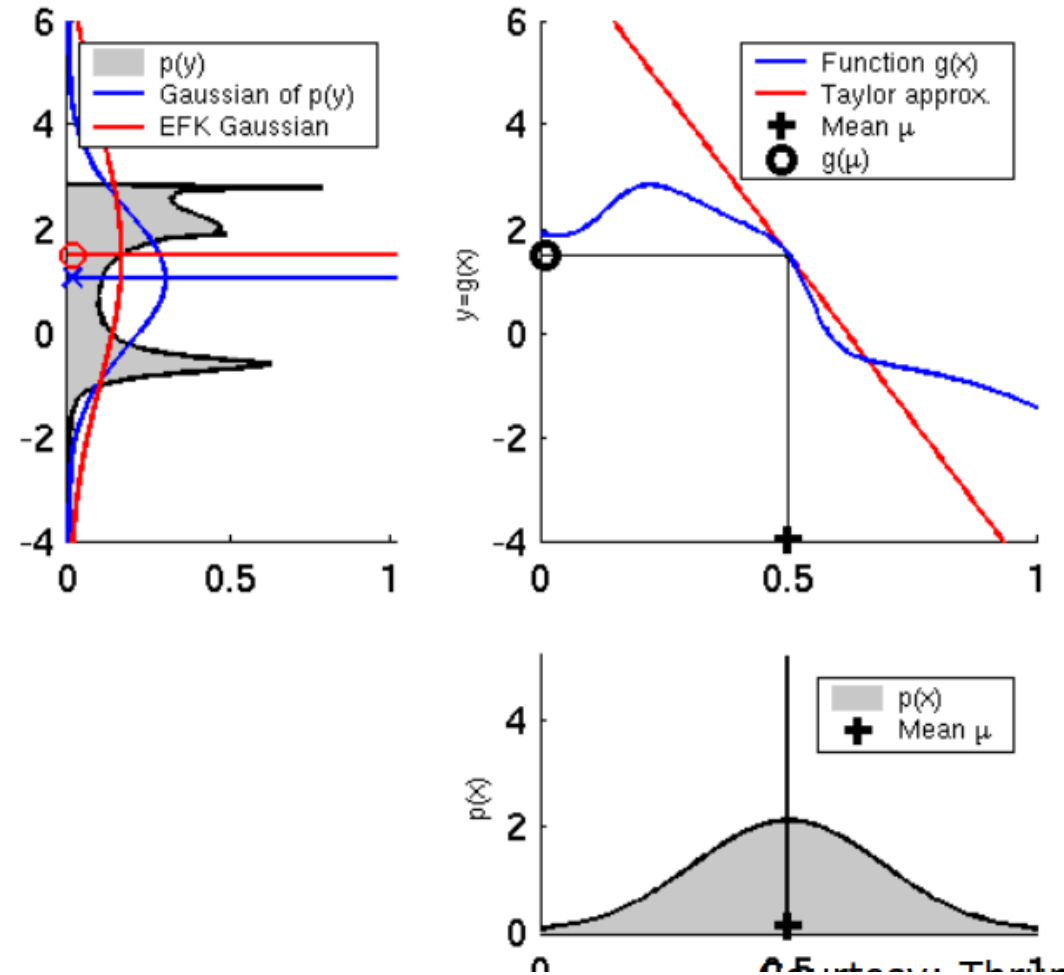


- Generalizes the gradient of a scalar valued function

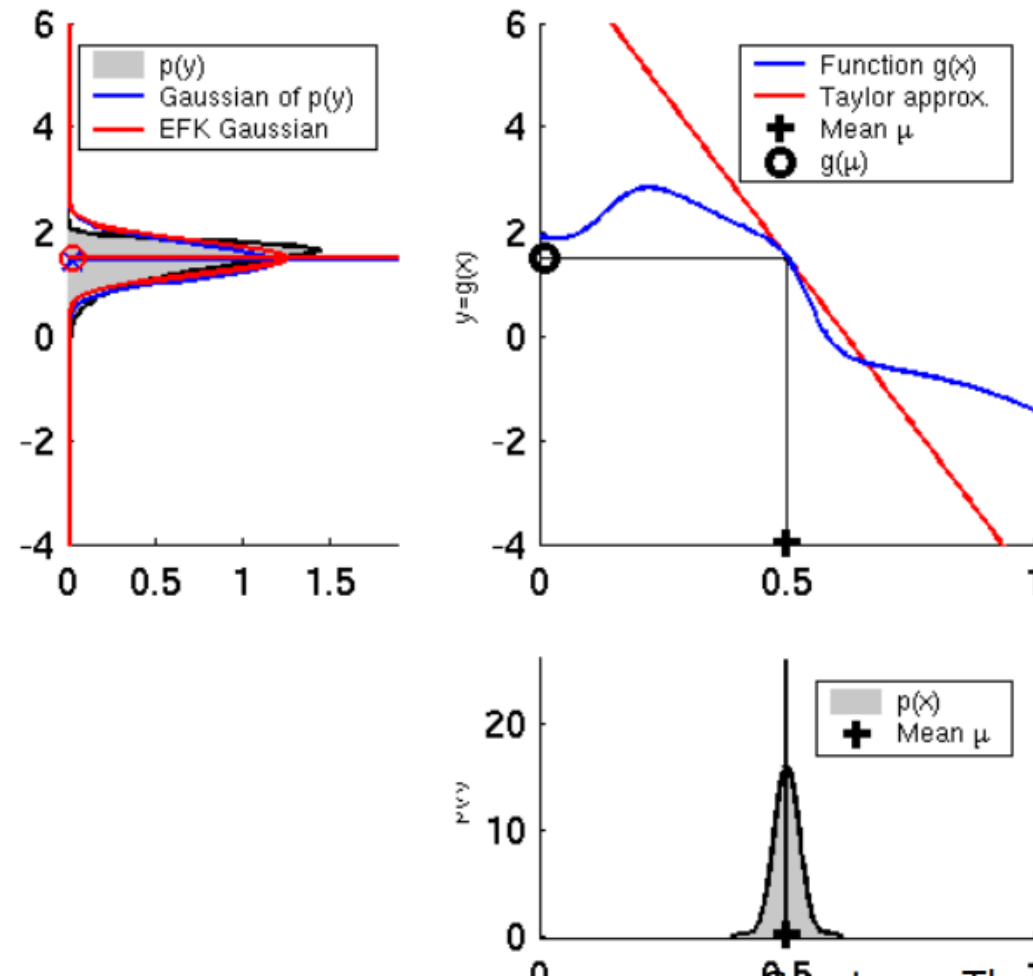
# EKF Linearization



# EKF Linearization



# EKF Linearization



# Linearized Motion Model

- The linearized motion model leads to

$$p(x_t | u_t, x_{t-1}) \approx \det(2\pi R_t)^{-1/2} \exp \left( -\frac{1}{2} (x_t - g(u_t, \mu_{t-1}) - G_t(x_{t-1} - \mu_{t-1}))^T R_t^{-1} (x_t - \underbrace{g(u_t, \mu_{t-1}) - G_t(x_{t-1} - \mu_{t-1})}_{\text{linearized model}}) \right)$$

- $R_t$  describes the noise of the motion

# Linearized Observation Model

- The linearized model leads to

$$p(z_t|x_{t-1}) \approx \det(2\pi R_t)^{-1/2} \exp \left( -\frac{1}{2} (z_t - h(\bar{\mu}_t) - H_t(x_t - \bar{\mu}_t))^T Q_t^{-1} (z_t - \underbrace{(h(\bar{\mu}_t) - H_t(x_t - \bar{\mu}_t))}_{\text{linearized model}}) \right)$$

- $Q_t$  describes the measurement noise

# Extended Kalman Filter Algorithm

1. Input:  $\mu_{t-1}, \Sigma_{t-1}, u_t, z_t$
  2.  $\bar{\mu}_t = g(u_t, \mu_{t-1})$
  3.  $\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^T + R_t$
  4.  $K_t = \bar{\Sigma}_t H_t^T (H_t \bar{\Sigma}_t H_t^T + Q_t)^{-1}$
  5.  $\mu_t = \bar{\mu}_t + K_t (z_t - h(\bar{\mu}_t))$
  6.  $\Sigma_t = (I - K_t H_t) \bar{\Sigma}_t$
  7. return:  $\mu_t, \Sigma_t$
- With the following substitutions:  $A_t \rightarrow G_t$     $C_t \rightarrow H_t$

# Kalman Filter Algorithm

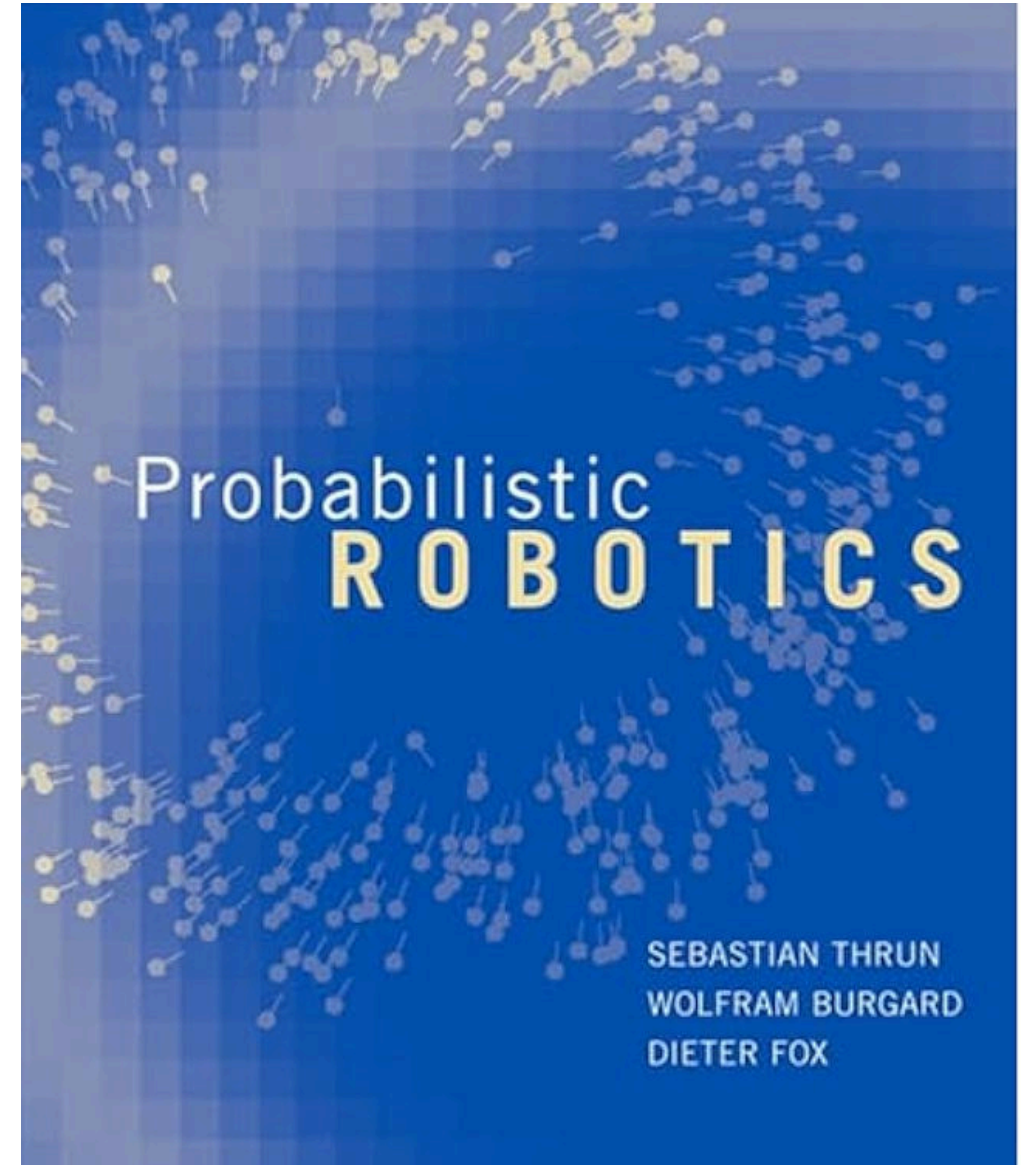
- Kalman filter is a Bayes filter for the linear Gaussian case
- All models/beliefs must be Gaussian
- Linear motion and observation model
- Optimal estimator
- Linear models are a serious limitation in real-world applications

# Extended Kalman Filter Summary

- Extension of the Kalman filter
- One way to handle the non-linearities
- Performs local linearizations (Taylor)
- Works well in practice for moderate nonlinearities
- Large uncertainty leads to increased approximation error

# Literature

- Thrun et al. "Probabilistic Robotics" Chapter 3.1 & 3.2 (Kalman Filter)
- Thrun et al. "Probabilistic Robotics" Chapter 3.3 (Extended Kalman Filter)



# Limitations of the Extended Kalman Filter

- The EKF is **NOT** optimal, its performance depends on how close we are to the linearization point and how much nonlinear are our functions
- Many times the states *live* on nonlinear manifolds:  $SO(3)$ ,  $SE(3)$
- If we could find a way to operate directly on the manifold and propagate the error on their tangent space, it would be better ...
- This is **critical** for uncertainty propagation

# Lie Group Theory

# Matrix Lie groups

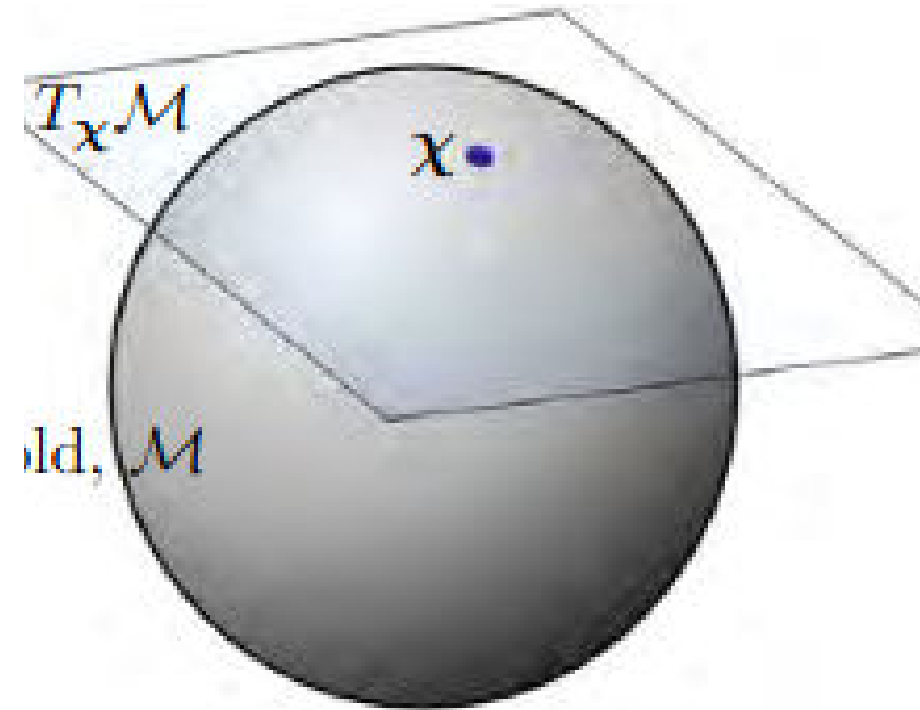
- A *matrix Lie group*  $\mathbf{G}$  is a subset of square invertible  $N \times N$  matrices  $\mathcal{M}_N(\mathbb{R})$  satisfying the following properties:

$$\mathbf{I}_N \in \mathbf{G} \quad \forall \mathbf{x} \in \mathbf{G}, \mathbf{x}^{-1} \in \mathbf{G} \quad \forall \mathbf{x}_1, \mathbf{x}_2 \in \mathbf{G}, \mathbf{x}_1 \mathbf{x}_2 \in \mathbf{G}$$

- To every point  $\mathbf{x} \in \mathbf{G}$  one can associate a *vector space*  $T_{\mathbf{x}}\mathbf{G}$  called the *tangent space at  $\mathbf{x}$* . The tangent space at the identity  $\mathbf{I}_N$  is called **Lie algebra**  $\mathfrak{g} \subset \mathcal{M}_N(\mathbb{R})$ . This is the subset of *skew symmetric* matrices.
- There is always an invertible map  $\mathbb{R}^d \rightarrow \mathfrak{g}$  mapping  $\xi \in \mathbb{R}^d$  to  $\xi^\wedge \in \mathfrak{g}$
- For any  $\xi \in \mathbb{R}^d$  and  $\mathbf{x} \in \mathbf{G}$ , the tangent space  $T_{\mathbf{x}}\mathbf{G}$  can be identified through a left  $(\xi^\wedge)\mathbf{x}$  or right multiplication  $\mathbf{x}(\xi^\wedge)$ , which are different because matrix multiplication is not commutative

# From Lie Algebra and back

- The *Lie exponential map*  $\exp_m : \mathfrak{g} \rightarrow \mathbf{G}$  is a bijection between a neighborhood of  $\mathfrak{g}$  at  $\mathbf{0}$  and a neighborhood of  $\mathbf{G}$  at  $\mathbf{I}_N$
- For convenience, we define the *exponential map*  $\exp : \mathbb{R}^d \rightarrow \mathbf{G}$  defined as  $\exp(\xi) = \exp_m(\xi^\wedge)$



# Uncertainty in Lie groups

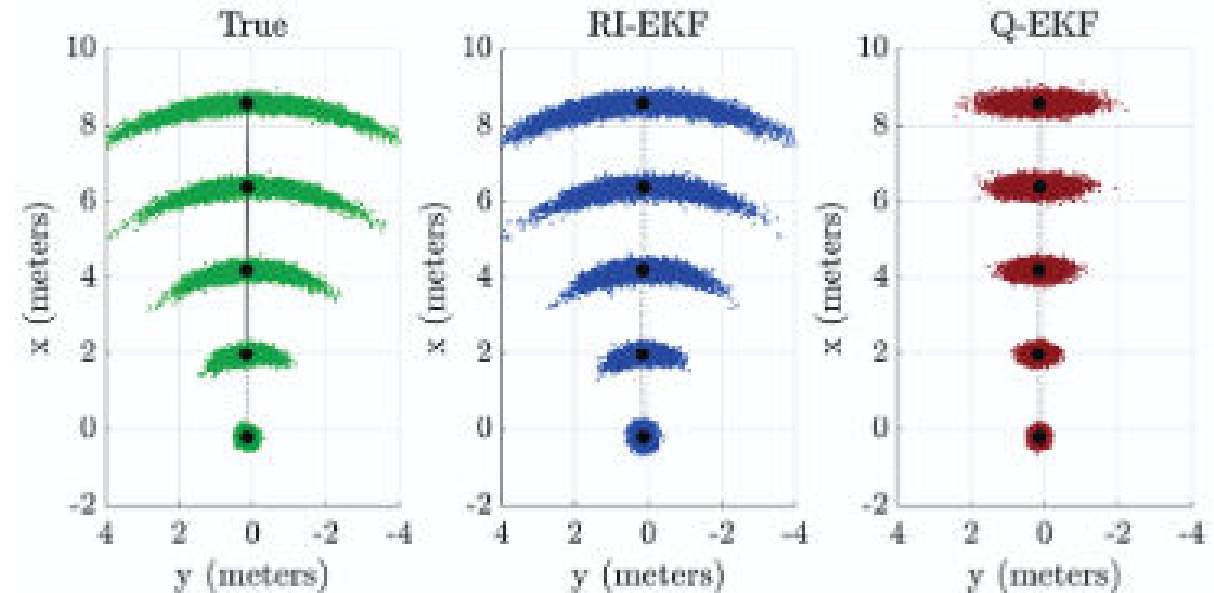
- To define random variables on Lie groups, we cannot apply the usual approach of additive noise for  $\mathbf{x} \in \mathbf{G}$ , because  $\mathbf{G}$  is not a vector space
- Instead, we define the probability distribution  $\mathbf{x} \sim \mathcal{N}_L(\bar{\mathbf{x}}, \Sigma)$  as follows:

$$\mathbf{x} = \bar{\mathbf{x}} \exp(\xi) \quad \xi \sim \mathcal{N}(0, \Sigma)$$

- $\mathcal{N}(0, \Sigma)$  is a regular Gaussian distribution on  $\mathbb{R}^d$  (which is  $\mathfrak{g}$ ) and  $\Sigma$  is the covariance matrix associated to the distribution
- $\mathcal{N}_L$  is a *concentrated Gaussian* that lives in  $\mathbf{G}$ .
- In the same way, we can define  $\mathcal{N}_R$  such as  $\mathbf{x} = \exp(\xi)\bar{\mathbf{x}} \quad \xi \sim \mathcal{N}(0, \Sigma)$

# Invariant Extended Kalman Filter

- The *Invariant EKF* exploits the properties of a system whose state is defined as matrix Lie Group  $SE_2(3)$
- This is critical for error propagation



# References

- Axel Barrau, Silvère Bonnabel. Invariant Kalman filtering. 2025. ⟨hal-05088150⟩
- Ross Hartley, Maani Ghaffari Jadidi, Jessy W. Grizzle, Ryan M. Eustice  
Contact-Aided Invariant Extended Kalman Filtering for Legged Robot State Estimation, IJRR

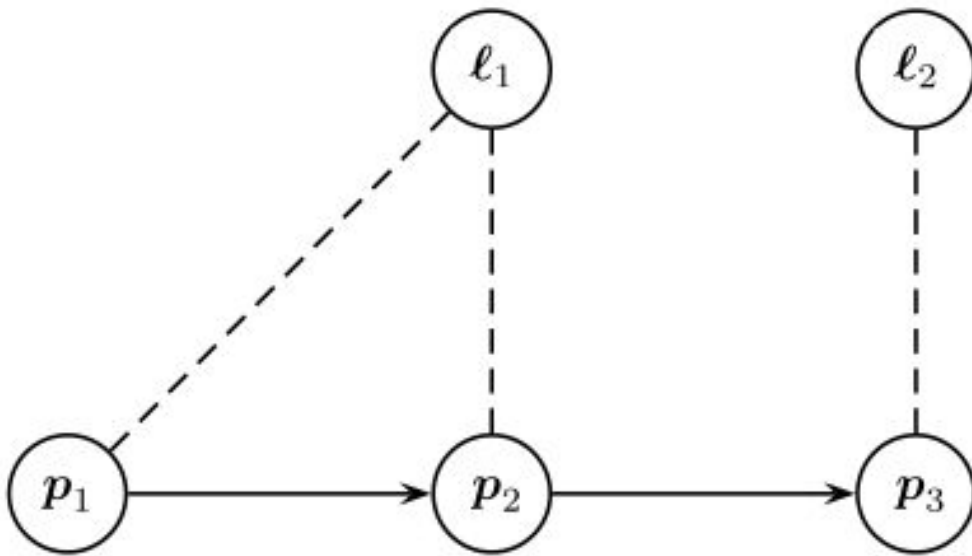
# Smoothing and Factor Graphs

- Need to estimate your state even in the past?
- Missing a process model?
- Many sources of measurement?
- We can solve state estimation through *optimization* using *factor graphs*

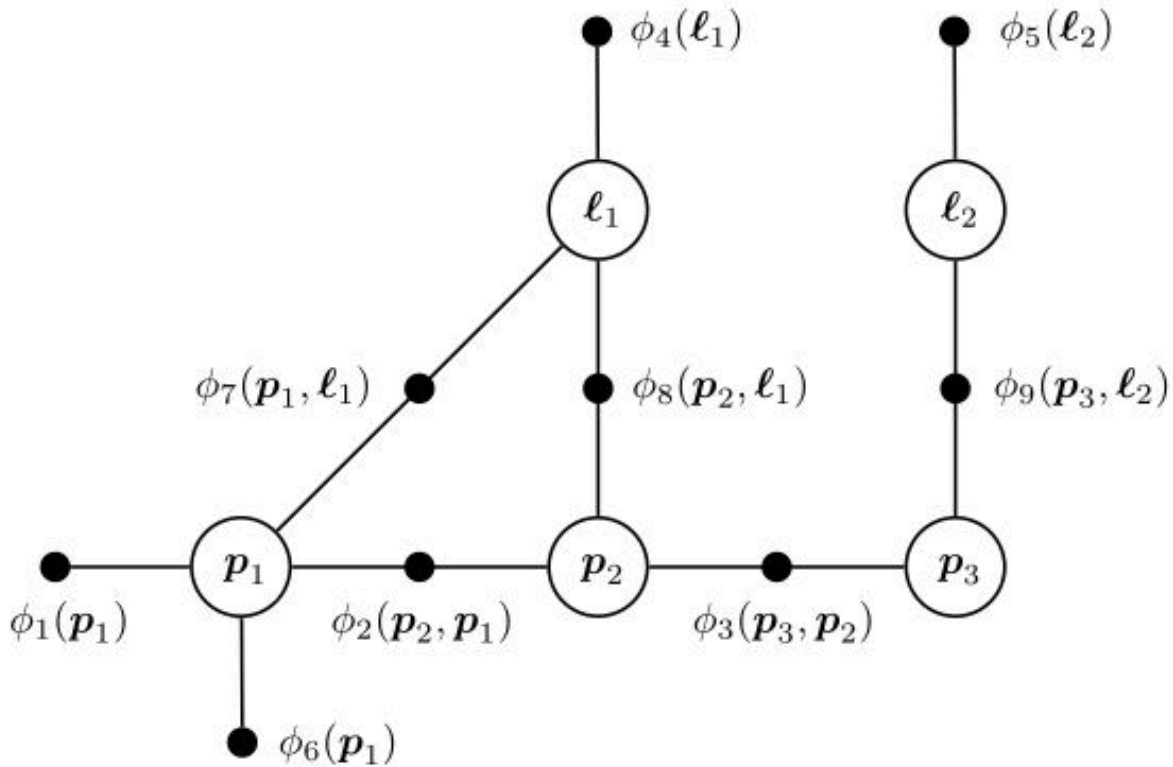
# What is a factor graph?

Let's consider a simple SLAM scenario with unknown state  $\mathbf{x} = [\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3, \ell_1, \ell_2]^\top$

The objective of our estimation is to reconstruct the probability density function  $p(\mathbf{x}|\mathbf{z})$



# Factor Graphs as graphical models



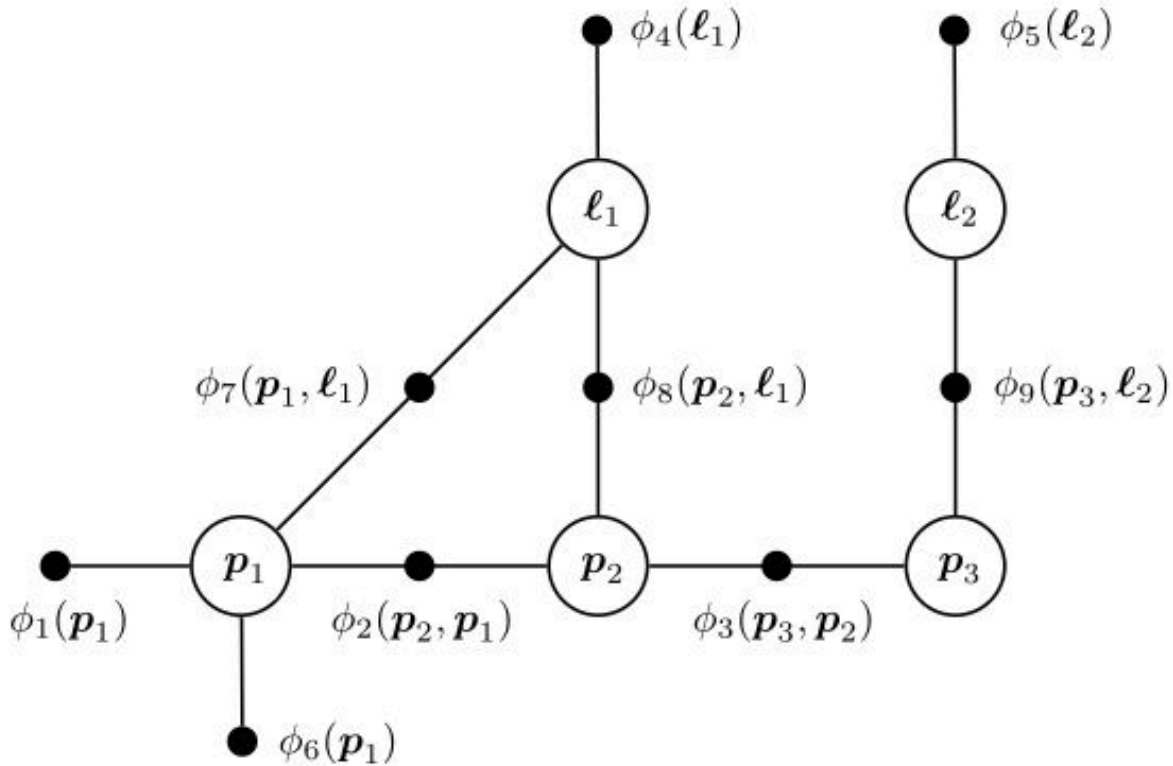
Factor graphs are *probabilistic graphical models* can be used to model a probability density as a product of factors.

They can model any factored function  $\phi(\mathbf{x})$

# Factor Graphs

Formally, they are *undirected bipartite graphs* in which:

- Named nodes refer to the unknowns of our problem
- Unnamed nodes refer to measurements
- Edges connect the former to the latter and viceversa

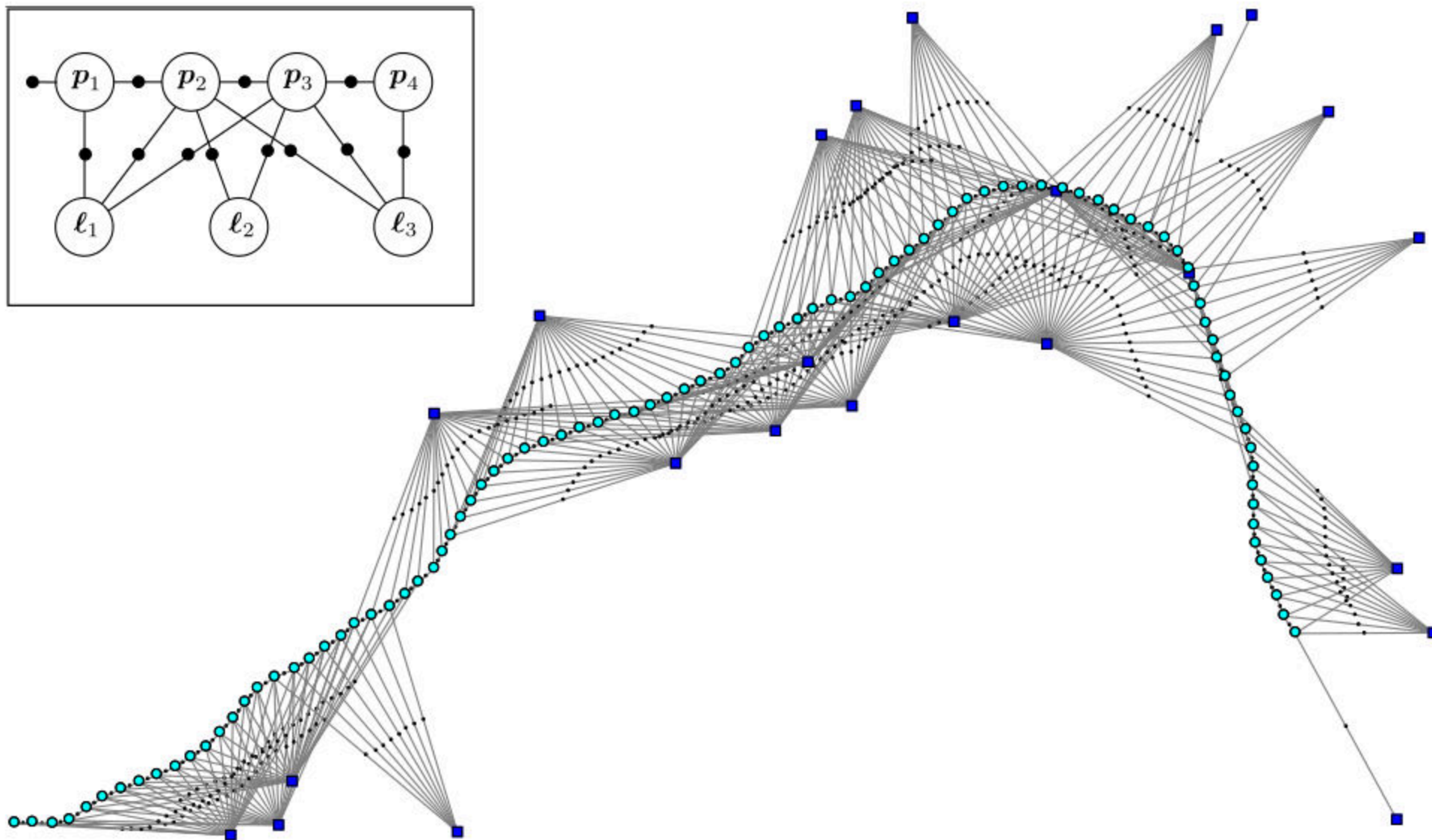
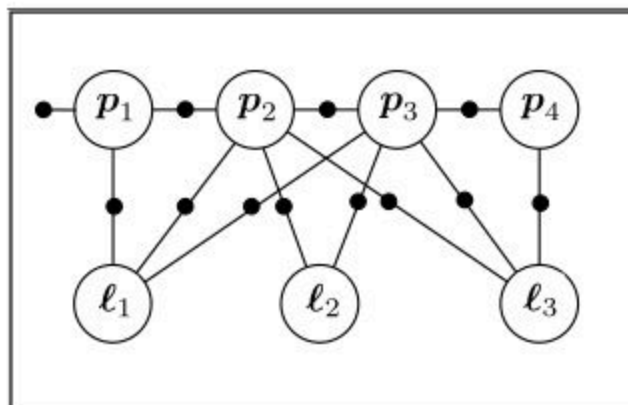


# Conditioned probability chain

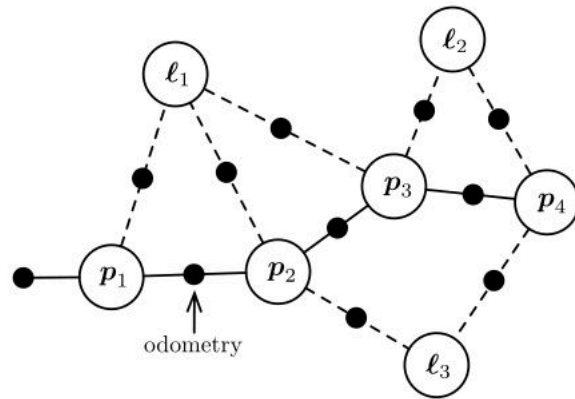
$$\begin{aligned} p(\boldsymbol{x} \mid \boldsymbol{z}) &\propto p(\boldsymbol{p}_1) p(\boldsymbol{p}_2 \mid \boldsymbol{p}_1) p(\boldsymbol{p}_3 \mid \boldsymbol{p}_2) \\ &\quad \times p(\boldsymbol{\ell}_1) p(\boldsymbol{\ell}_2) \\ &\quad \times p(\boldsymbol{z}_1 \mid \boldsymbol{p}_1) \\ &\quad \times p(\boldsymbol{z}_2 \mid \boldsymbol{p}_1, \boldsymbol{\ell}_1) p(\boldsymbol{z}_3 \mid \boldsymbol{p}_2, \boldsymbol{\ell}_1) p(\boldsymbol{z}_4 \mid \boldsymbol{p}_3, \boldsymbol{\ell}_2) \end{aligned}$$

# Equivalent factor product

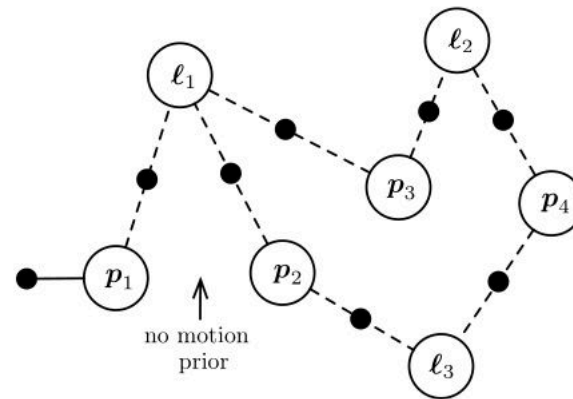
$$\begin{aligned}\phi(\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3, \boldsymbol{\ell}_1, \boldsymbol{\ell}_2) &= \phi_1(\mathbf{p}_1) \phi_2(\mathbf{p}_2, \mathbf{p}_1) \phi_3(\mathbf{p}_3, \mathbf{p}_2) \\ &\quad \times \phi_4(\boldsymbol{\ell}_1) \phi_5(\boldsymbol{\ell}_2) \\ &\quad \times \phi_6(\mathbf{p}_1) \\ &\quad \times \phi_7(\mathbf{p}_1, \boldsymbol{\ell}_1) \phi_8(\mathbf{p}_2, \boldsymbol{\ell}_1) \phi_9(\mathbf{p}_3, \boldsymbol{\ell}_2)\end{aligned}$$



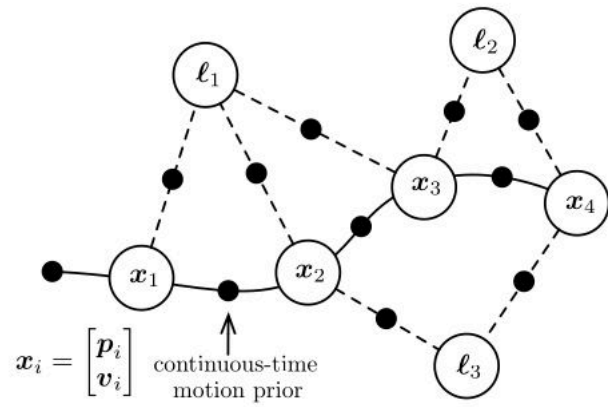
# A factor graph for every problem



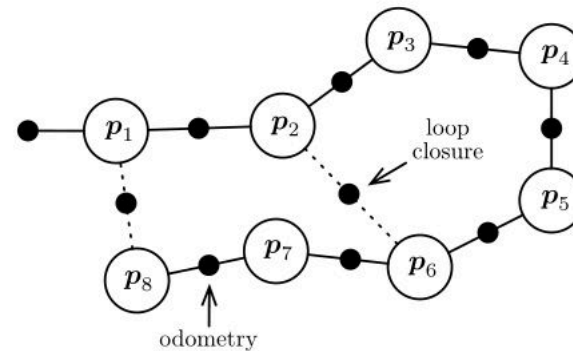
canonical landmark-based SLAM



bundle adjustment (BA)  
(structure from motion)



simultaneous trajectory estimation  
and mapping (STEAM)



pose-graph optimization (PGO)  
(pose-graph SLAM)

# Maximum-a-posteriori

$$\begin{aligned}\boldsymbol{x}^{\text{MAP}} &= \arg \max_{\boldsymbol{x}} p(\boldsymbol{x} \mid \boldsymbol{z}) \\ &= \arg \max_{\boldsymbol{x}} \frac{p(\boldsymbol{z} \mid \boldsymbol{x})p(\boldsymbol{x})}{p(\boldsymbol{z})} \\ &= \arg \max_{\boldsymbol{x}} p(\boldsymbol{z} \mid \boldsymbol{x})p(\boldsymbol{x})\end{aligned}$$

# MAP modeled by the factor graph

$$\phi(\boldsymbol{x}) = \prod_i \phi_i(\boldsymbol{x}_i)$$

$$\boldsymbol{x}^{\text{MAP}} = \arg \max_{\boldsymbol{x}} \phi(\boldsymbol{x})$$

how to model the measurement model and prior densities?

# Summary

- Factor graphs represent the factorization of a probability function.
- Nodes: variables; factors: constraints between variables.
- Widely used in SLAM back-end optimization.

# Multivariate Gaussian Density

$$\mathcal{N}(\boldsymbol{\theta}, \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{\sqrt{|2\pi\boldsymbol{\Sigma}|}} \exp\left(-\frac{1}{2}\|\boldsymbol{\theta} - \boldsymbol{\mu}\|_{\boldsymbol{\Sigma}}^2\right)$$

$$p(\boldsymbol{z}|\boldsymbol{p}, \boldsymbol{\ell}) = \mathcal{N}(\boldsymbol{z}, \boldsymbol{h}(\boldsymbol{p}, \boldsymbol{\ell}), \boldsymbol{\Sigma}_R) = \frac{1}{\sqrt{|2\pi\boldsymbol{\Sigma}_R|}} \exp\left(-\frac{1}{2}\|\boldsymbol{z} - \boldsymbol{h}(\boldsymbol{p}, \boldsymbol{\ell})\|_{\boldsymbol{\Sigma}_R}^2\right)$$

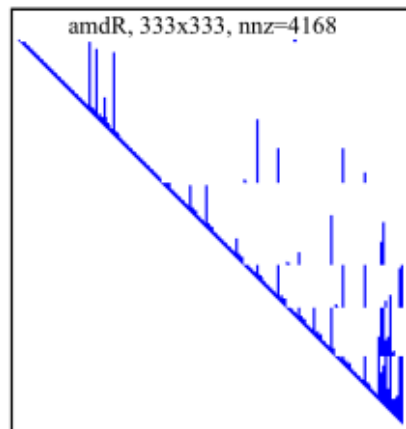
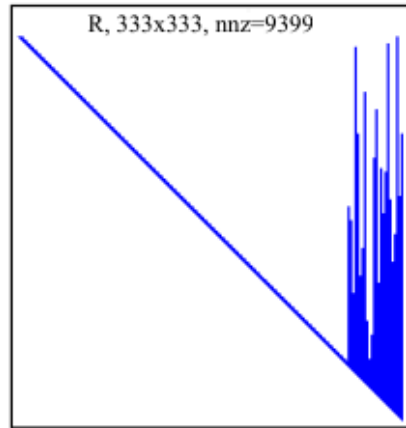
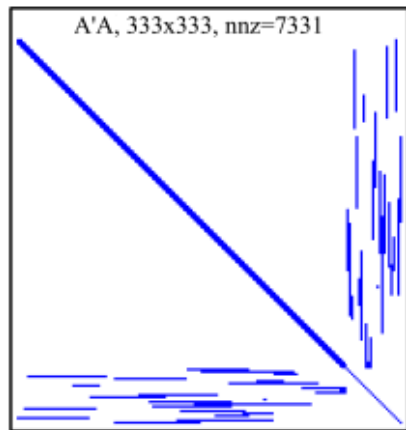
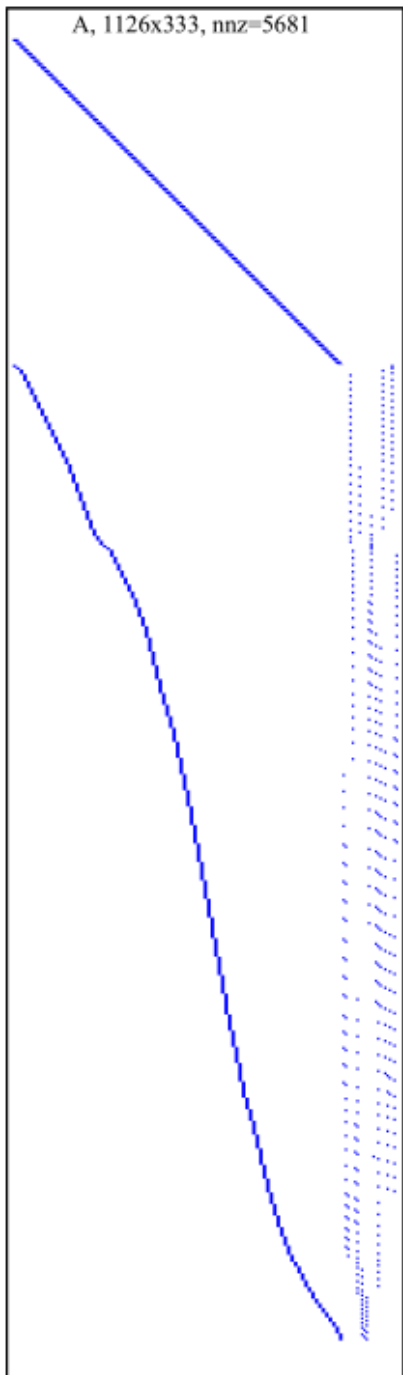
# Nonlinear Least Squares

$$\phi_i(\mathbf{x}_i) \propto \exp \left( -\frac{1}{2} \|\mathbf{z}_i - \mathbf{h}_i(\mathbf{x}_i)\|_{\Sigma_i}^2 \right)$$

$$\mathbf{x}^{\text{MAP}} = \arg \min_{\mathbf{x}} \sum_i \|\mathbf{z}_i - \mathbf{h}_i(\mathbf{x}_i)\|_{\Sigma_i}^2$$

# Nonlinear Optimization

1. Initialize state estimate.
2. Linearize residuals around current estimate.
3. Solve the linearized system.
4. Update state.
5. Repeat until convergence.



# Sparsity matters

Different factorizations produce the same results but with a different level of sparsity, hence efficiency.

- *Left:* Jacobian matrix  $\mathbf{A}$  with 333 unknowns
- *Right top:* Information matrix  $\mathbf{\Lambda} = \mathbf{A}^T \mathbf{A}$  with 7k nnz
- *Right middle:* Cholesky triangle  $\mathbf{R}$  9k nnz
- *Right bottom:* COLAMD ordering 4k nnz

# Summary

- Linearized optimization in SLAM can be stated with linear algebra
- Factorize either information matrix or measurement Jacobian into Square Root Form
- The family of approaches is called *square-root* SAM

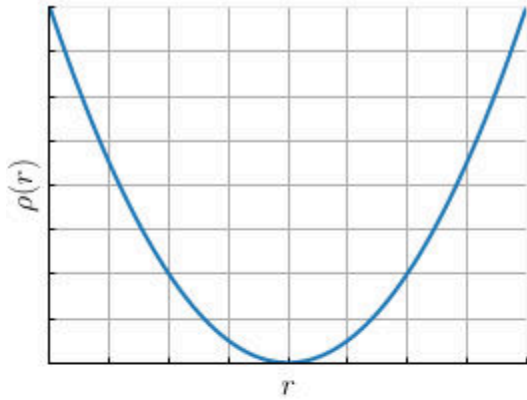
# M-Estimators

At the backend, we can replace the squared loss with a robust loss.

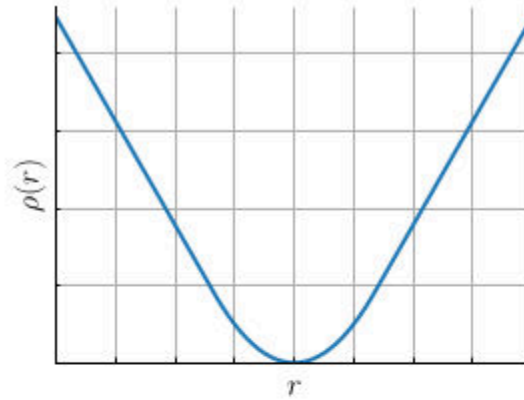
$$\boldsymbol{x}^{\text{MAP}} = \arg \min_{\boldsymbol{x}} \sum_i \boldsymbol{r}_i(\boldsymbol{x}_i)^2 \implies \boldsymbol{x}^{\text{MEST}} = \arg \min_{\boldsymbol{x}} \sum_i \rho(\boldsymbol{r}_i(\boldsymbol{x}_i))$$

The robust loss function  $\rho(\cdot)$  has to be non-negative and have a derivative much smaller than  $2\boldsymbol{r}$  when  $\boldsymbol{r}$  is large, so that it doesn't influence the gradient descent

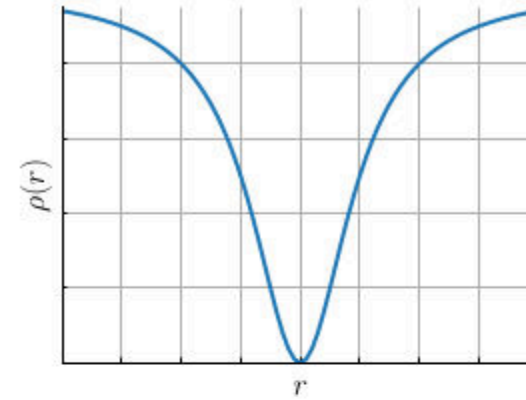
# A menu of robust cost functions ...



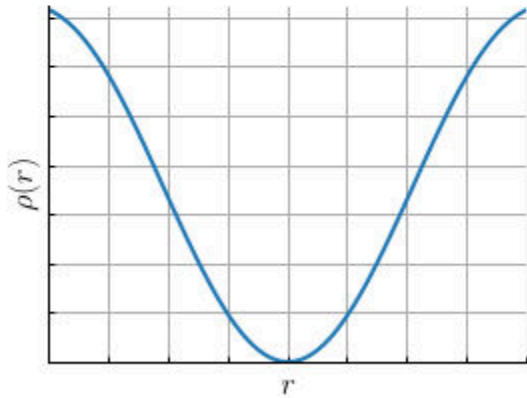
(a) Quadratic loss



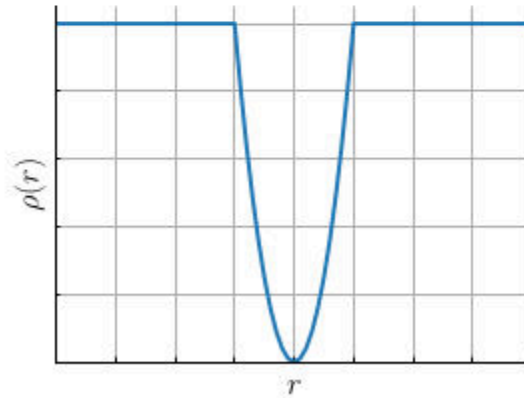
(b) Huber loss



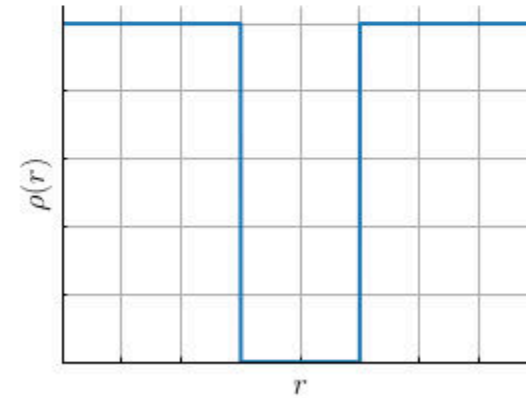
(c) Geman-McClure



(d) Tukey's biweight



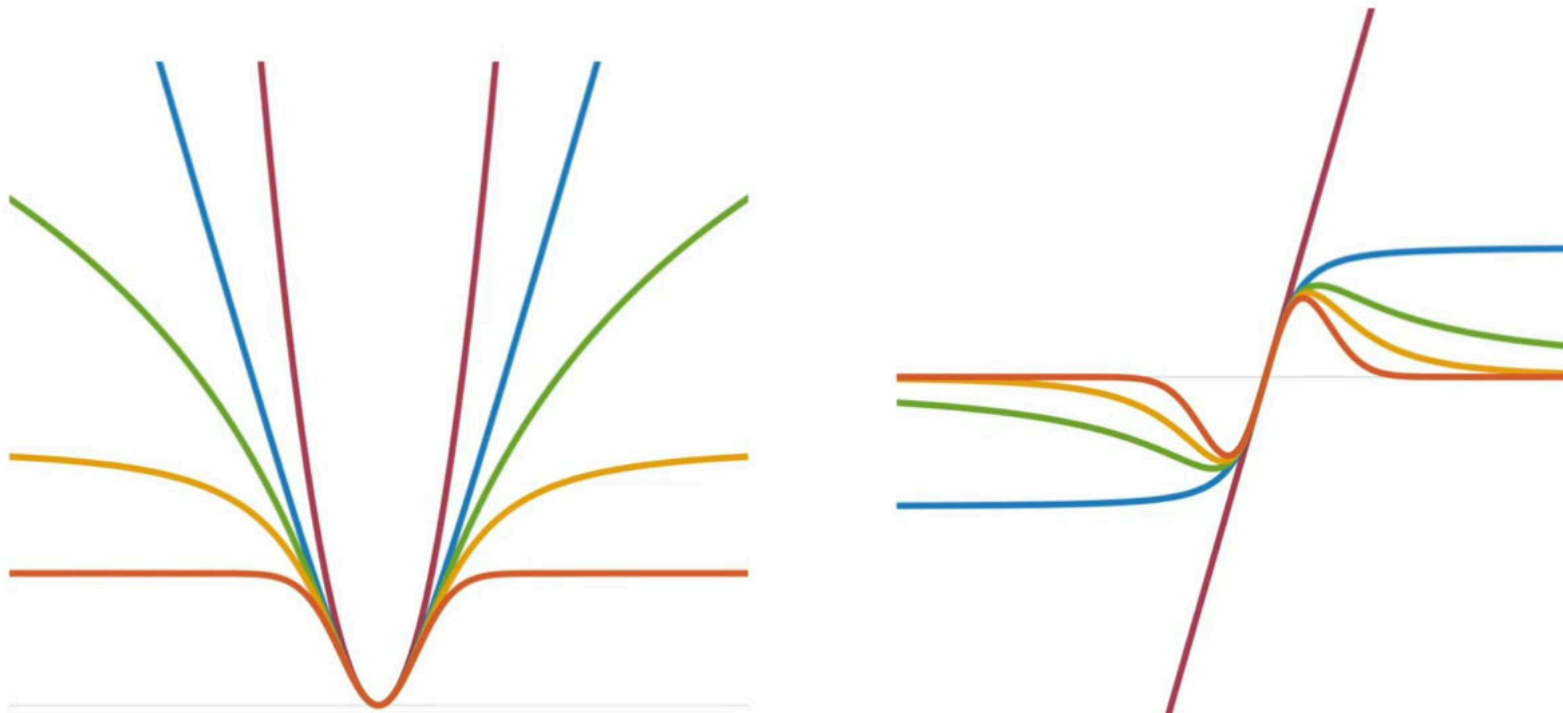
(e) Truncated Quadratic



(f) Maximum Consensus loss

## ... which can be generalized

In 2019 Jon Barron found a superset of all robust cost functions parametrized by  $\alpha$



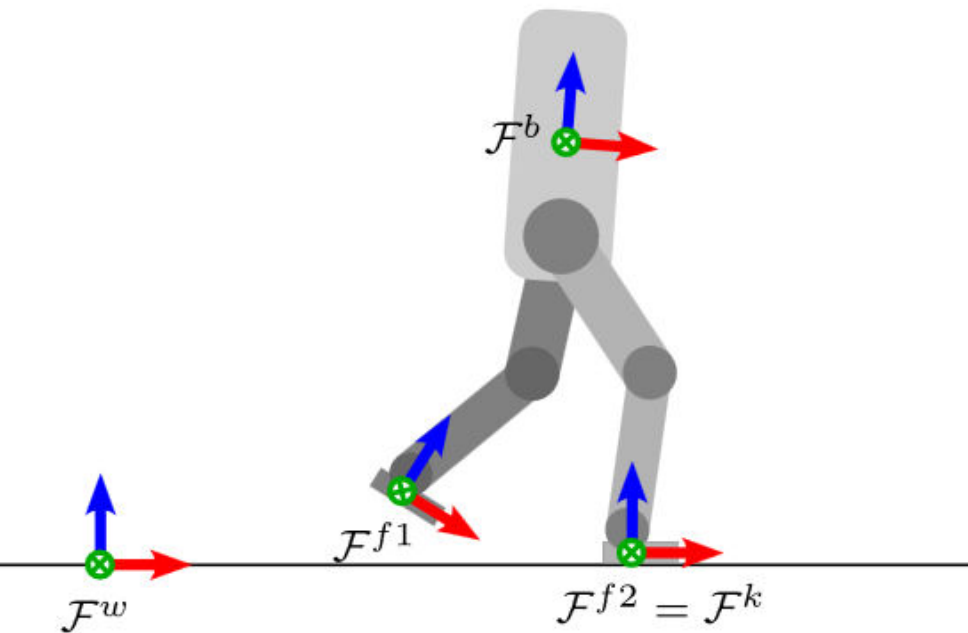
Video here:

<https://www.youtube.com/watch?v=BmNKbnF69eY>

# Inertial-Legged State Estimation

# Leg odometry

- *Leg odometry* is the process of estimating the incremental motion of the robot from leg kinematics (akin to wheel odometry for rovers)
- Legged robots have intermittent contact with the ground
- Each leg stride has an *aerial* phase and a *stance* phase
- The *stance* feet are the ones propelling the robot
- Leg odometry reduces to two subproblems:
  - Establish what feet are in stance (*contact estimation*)
  - Determine the incremental motion from stance feet (*motion estimation*)

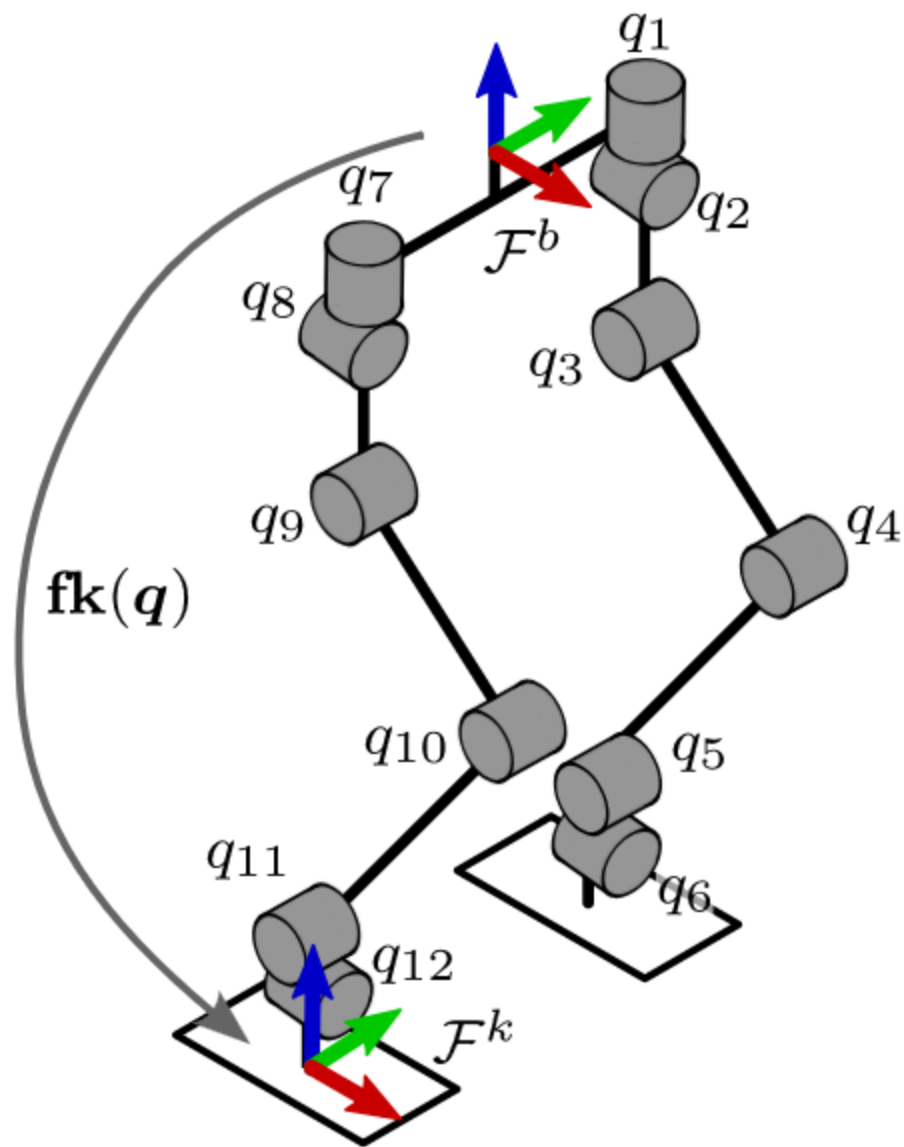
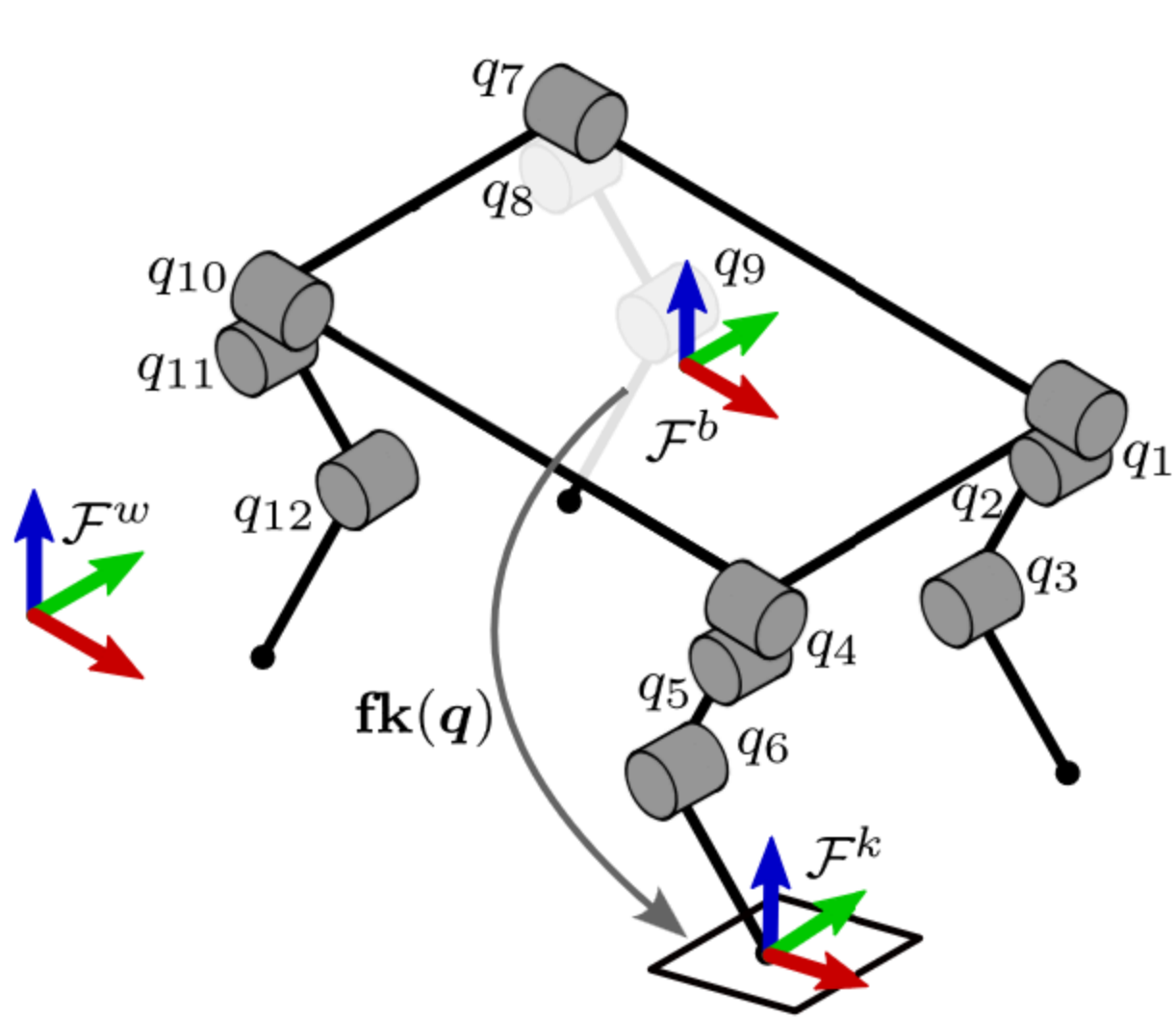


## Reference frames

- World frame  $\mathcal{F}^w$  (inertial, earth-fixed)
- Base frame  $\mathcal{F}^b$  (floating, robot-fixed)
- Foot frames  $\mathcal{F}^{f1}, \mathcal{F}^{f2}$  (feet-fixed)
- Contact frames  $\mathcal{F}^k$ , coincident to foot frames when in stance

# The state

- We are interested in knowing *position, orientation, and velocity* of the robot
- We assume an IMU can measure *angular velocity*  $\boldsymbol{\omega}$  and *proper acceleration*  $\boldsymbol{a}$
- Its measurements are affected by Gaussian noise and **bias**
- Hence, the state at time  $t$  is typically defined as:  $\boldsymbol{x}_t = [\mathbf{p}_t, \mathbf{R}_t, \mathbf{v}_t, \mathbf{b}_t^a, \mathbf{b}_t^\omega]$



# Legged Robot Kinematics

- The *forward kinematics* function  $\mathbf{fk}(\mathbf{q}) : \mathbb{R}^{12} \rightarrow \text{SE}(3)$  gives the pose of  $f$  in  $b$

$$\mathbf{T}_f^b = \mathbf{fk}(\mathbf{q}) = \begin{bmatrix} \mathbf{f}_R(\mathbf{q}) & \mathbf{f}_p(\mathbf{q}) \\ \mathbf{0} & 1 \end{bmatrix}$$

- The *Jacobian matrix*  $\mathbf{J}(\mathbf{q}) : \mathbb{R}^{12} \rightarrow \mathbb{R}^{6 \times 12}$  gives the foot velocity in  $b$

$$\begin{bmatrix} \mathbf{v}_f^b \\ \boldsymbol{\omega}_f^b \end{bmatrix} = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}} = \begin{bmatrix} \mathbf{J}_v(\mathbf{q}) \\ \mathbf{J}_\omega(\mathbf{q}) \end{bmatrix} \dot{\mathbf{q}}$$

- Each leg is kinematically independent, therefore  $\mathbf{J}(\mathbf{q})$  is a sparse block matrix

# Legged Robot Dynamics

- The dynamics of a legged robot can be expressed as two coupled dynamics equations, computed using the Recursive Newton-Euler algorithm

$$\mathbf{M}(\mathbf{q}) \begin{bmatrix} \dot{\mathbf{v}} \\ \dot{\boldsymbol{\omega}} \\ \ddot{\mathbf{q}} \end{bmatrix} + \mathbf{h}(\mathbf{q}, \dot{\mathbf{q}}) = \begin{bmatrix} \mathbf{J}_b^T \\ \mathbf{J}_q^T \end{bmatrix} \mathbf{f} + \begin{bmatrix} \mathbf{0}_6 \\ \boldsymbol{\tau} \end{bmatrix}$$

- $\mathbf{M}(\mathbf{q})$  is the *mass matrix*,  $\mathbf{h}(\mathbf{q}, \dot{\mathbf{q}})$  is the bias term,  $\boldsymbol{\tau}$  are the active torques
- $d$  is the number of DoF per leg,  $c$  the number of legs in contact
- $\mathbf{J}_b(\mathbf{q}) \in \mathbb{R}^{dc \times 6}$  is the Jacobian mapping base twist to feet velocities
- $\mathbf{J}_q(\mathbf{q}) \in \mathbb{R}^{dc \times 12}$  is the stack of non-zero Jacobian blocks of the legs in contact
- $\mathbf{f}$  is the stack of forces and torques acting on the contact point

$$M(\mathbf{q}) \begin{bmatrix} \ddot{\mathbf{q}}_b \\ \ddot{\mathbf{q}} \end{bmatrix} + \mathbf{h}(\mathbf{q}, \dot{\mathbf{q}}) = \begin{bmatrix} \mathbf{J}_b^T \\ \mathbf{J}_q^T \end{bmatrix} \mathbf{f} + \begin{bmatrix} \mathbf{0}_6 \\ \boldsymbol{\tau} \end{bmatrix}$$

$$\begin{array}{c}
 \begin{array}{ccc}
 6 \times 6 & 6 \times 12 & 6 \\
 \begin{array}{|c|} \hline I_c \\ \hline \end{array} & \begin{array}{|c|} \hline F \\ \hline \end{array} & \begin{array}{|c|} \hline \ddot{\mathbf{q}}_b \\ \hline \end{array} \\
 \begin{array}{|c|} \hline F^T \\ \hline \end{array} & \begin{array}{|c|} \hline M \\ \hline \end{array} & \begin{array}{|c|} \hline \ddot{\mathbf{q}} \\ \hline \end{array} \\
 12 \times 6 & 12 \times 12 & 12
 \end{array}
 + \begin{array}{c}
 6 \\
 \begin{array}{|c|} \hline h_b \\ \hline \end{array} \\
 12
 \end{array}
 = \begin{array}{c}
 6 \times 3c \\
 \begin{array}{|c|} \hline \mathbf{J}_b^T \\ \hline \end{array} \\
 \begin{array}{|c|} \hline \mathbf{J}_q^T \\ \hline \end{array} \\
 12 \times 3c
 \end{array}
 + \begin{array}{c}
 \begin{array}{|c|} \hline \mathbf{f} \\ \hline \end{array} \\
 3c
 \end{array}
 + \begin{array}{c}
 6 \\
 \begin{array}{|c|} \hline \mathbf{0} \\ \hline \end{array} \\
 \begin{array}{|c|} \hline \boldsymbol{\tau} \\ \hline \end{array} \\
 12
 \end{array}
 \end{array}$$

# **Sensors for Inertial-Legged State Estimation**

# Inertial Measurement Units



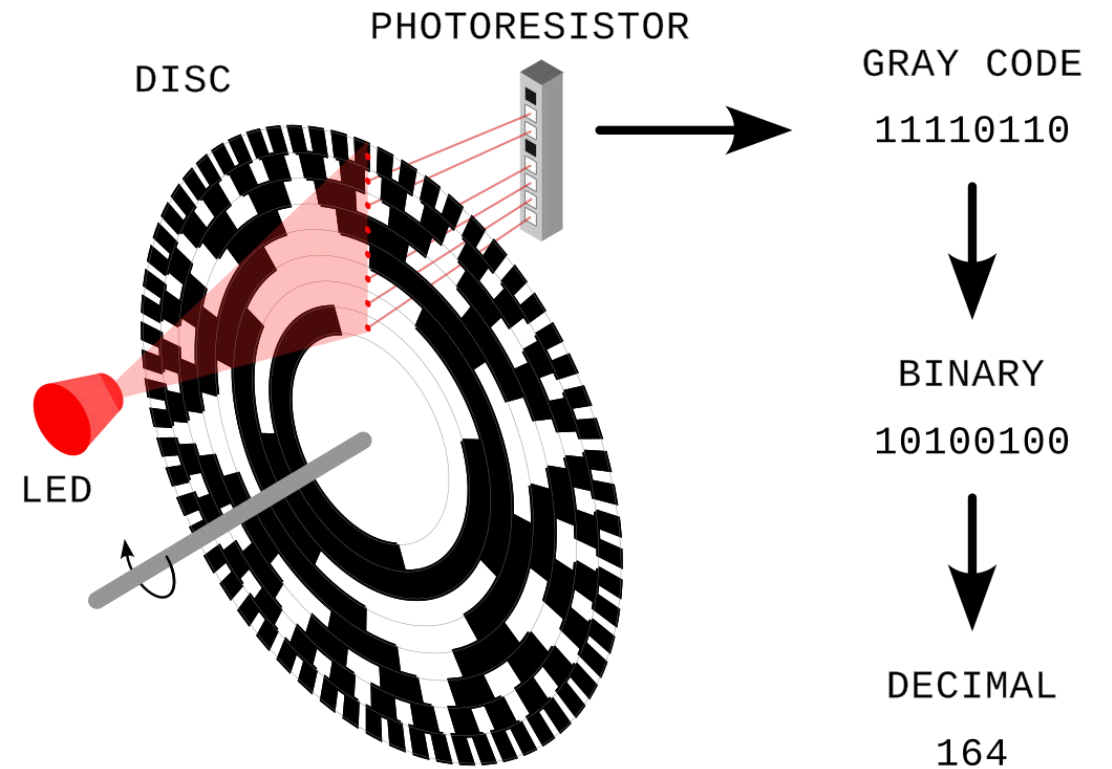
- They measure *angular velocity* and *proper acceleration* (give  $\mathbf{g}$  when still) on 3 axes
- Measurements affected by zero-mean Gaussian noise and random walk bias ( $\dot{\mathbf{b}} = \boldsymbol{\eta} \sim \mathcal{N}$ )
- Some models have:
  - a magnetometer for absolute yaw direction
  - internal filter that estimates the orientation

$$\tilde{\boldsymbol{\omega}} = \boldsymbol{\omega} - \mathbf{b}_{\omega} + \boldsymbol{\eta}_{\omega} \quad \boldsymbol{\eta}_{\omega} \sim \mathcal{N}(0, \Sigma_{\omega})$$

$$\tilde{\mathbf{a}} = \mathbf{R}^T(\mathbf{a} - \mathbf{g}^w) + \mathbf{b}_a + \boldsymbol{\eta}_a \quad \boldsymbol{\eta}_a \sim \mathcal{N}(0, \Sigma_a)$$

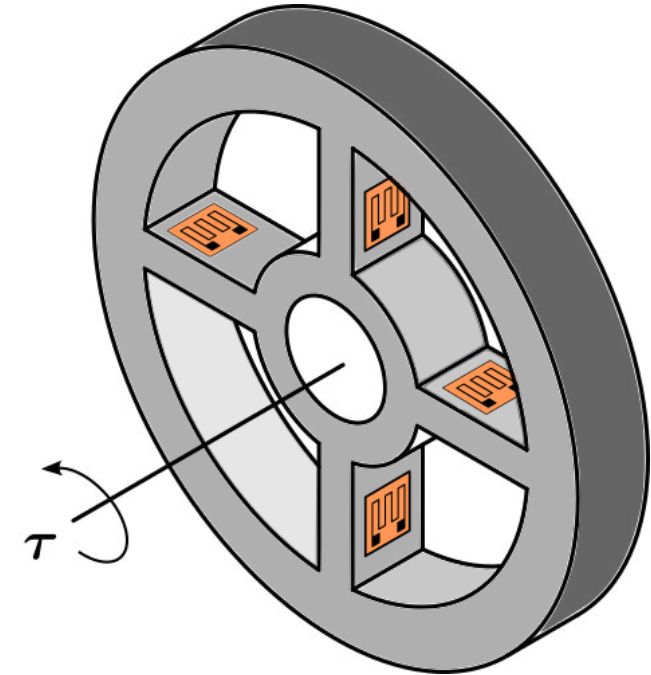
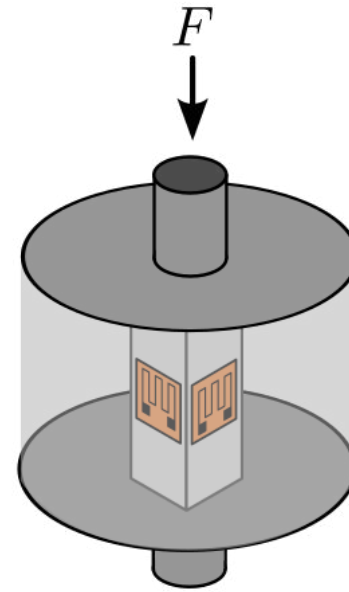
# Rotary Encoders

- Different ways to categorize
  - Absolute vs. Relative
  - Digital vs. Analog
  - Optical vs. Magnetic
- Most common now are:  
absolute, digital, magnetic
- Gray codes are used



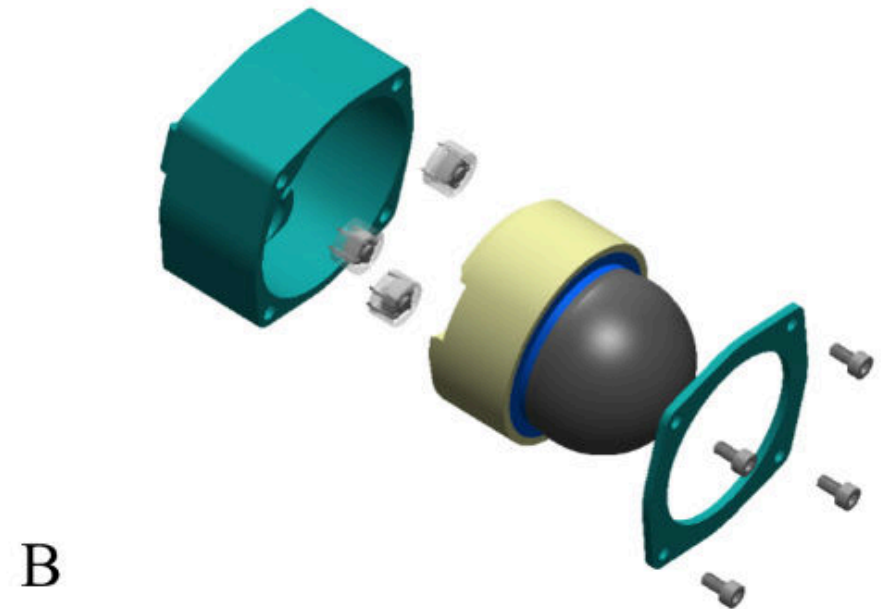
# Force / Torque Sensors

- Flexible variable resistor, *strain gauge* is glued to deformable elements of the structure
- If the motor is backdrivable, the *current* is proportional to the torque, so it can be inferred from it



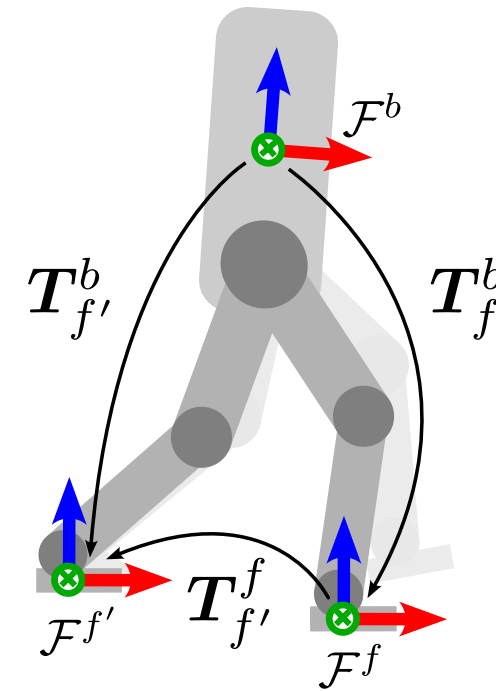
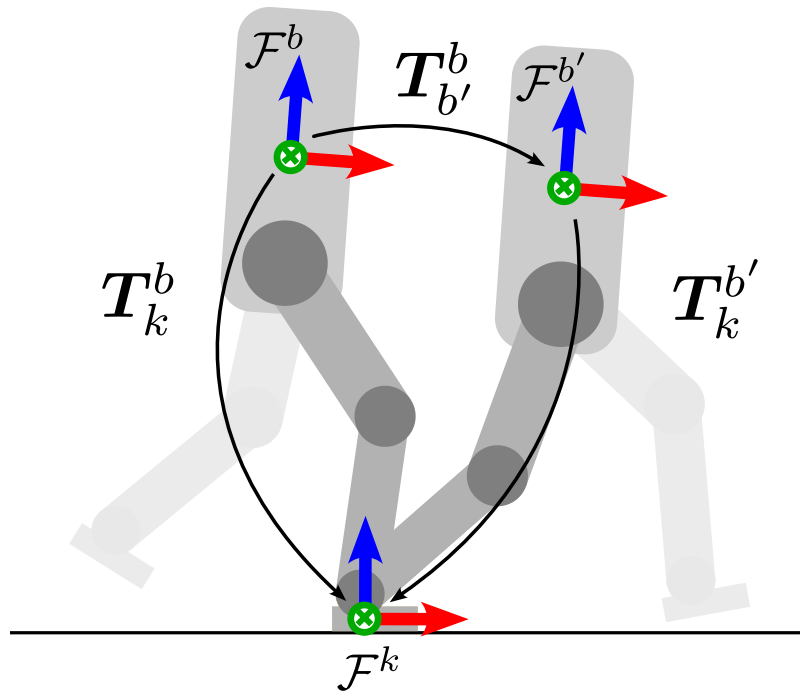
# Contact sensors

- Different ways to detect contact on the foot:
  - Mechanical switches
  - Force/Torque sensors
  - Pressure sensors
  - Optical sensors
  - IMUs
- All of the above have delays and non-idealities ...



# Motion Estimation

- If one foot frame is stationary, forward kinematics provides the displacement of the base over time. For quadrupeds, only position can be propagated.



$$\mathbf{T}_{b'}^b = \mathbf{T}_k^b (\mathbf{T}_k^{b'})^{-1} = (\mathbf{T}_{f'}^f)^{-1} = (\mathbf{T}_{f'}^f)^{-1} (\mathbf{T}_{f'}^f) = \mathbf{fk}(\mathbf{q}')^{-1} \mathbf{fk}(\mathbf{q})$$

- The above relation cannot be concatenated to do *dead reckoning* if we have flight phases or quadruped with point feet.
- The standard approach first introduced by (Bloesch et al. 2012) is to add the contact positions to the state (and orientations for humanoids)
- The state then becomes:
  - $\mathbf{x}_k = [\mathbf{p}_k, \mathbf{R}_k, \mathbf{v}_k, \mathbf{b}_k^a, \mathbf{b}_k^\omega, \mathbf{c}_1, \mathbf{c}_2, \mathbf{c}_3, \mathbf{c}_4, ]$  for quadrupeds
  - $\mathbf{x}_k = [\mathbf{p}_k, \mathbf{R}_k, \mathbf{v}_k, \mathbf{b}_k^a, \mathbf{b}_k^\omega, \mathbf{c}_1, \mathbf{c}_2, \mathbf{B}_1, \mathbf{B}_2, ]$  for humanoids

# Measurement model: position/pose estimation

The motion for an arbitrary leg  $i$  is:

$$\mathbf{T}_k^b = \mathbf{fk}(\mathbf{q}) = \mathbf{T}^{-1} \mathbf{C}_i$$

where

$$\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ \mathbf{0} & 1 \end{bmatrix} \quad \mathbf{C}_i = \begin{bmatrix} \mathbf{B}_i & \mathbf{c}_i \\ \mathbf{0} & 1 \end{bmatrix}$$

the following relationships then hold:

$$\mathbf{f}_p(\mathbf{q}) = \mathbf{R}^T (\mathbf{c}_i - \mathbf{p}) \quad \mathbf{f}_R(\mathbf{q}) = \mathbf{R}^\top \mathbf{B}_i$$

The rotation constraint is often ignored, for quadrupeds it is not available

# Measurement model: velocity

- The velocity at the contact point must be zero  $\mathbf{v}_k^w = \mathbf{0}$
- Then, velocity of the foot in contact/foot frame is the same and can be computed from the joint velocities:

$$\mathbf{v}_k^b = \mathbf{v}_f^b = \mathbf{J}_v(\mathbf{q})\dot{\mathbf{q}}$$

- The angular velocity is measured by the IMU, so we focus on the linear one
- The linear velocity of the robot becomes:

$$\mathbf{v}_b^w = -\boldsymbol{\omega}_b^w \times \mathbf{f}_p(\mathbf{q}) - \mathbf{J}_v(\mathbf{q})\dot{\mathbf{q}}$$

- We can express velocity in body frame or world frame by rotating it by  $\mathbf{R}_b^w$

# Contact Estimation

- With contact sensors, you get a binary measurements already
- With force/torque sensors, you have a force
- With IMU, you can detect when acceleration is zero
- Without any sensor at the feet, you can use the dynamics:

$$\mathbf{f}_i = -(\mathbf{J}^\top)^{-1}(\boldsymbol{\tau}_i - \mathbf{h}_{q,i} - \mathbf{F}^\top \dot{\mathbf{v}})$$

where  $\mathbf{J}$  is the nonzero block of leg  $i$  of  $\mathbf{J}_v$

# How to build process and sensor model?

- First, we need to separate the noise from the measurement and prove it can be modeled as additive zero-mean Gaussian
- We define the *state* and system dynamics
- We choose whether we want to add *position* or *velocity* measurements models
- If you want to do:
  - Kalman Filtering → define the prediction and correction steps
  - Smoothing → turn the process/measurements into *factors*

# Noise modeling

Measured encoder readings are affected by zero-mean Gaussian noise.

$$\tilde{\mathbf{q}} = \mathbf{q} + \boldsymbol{\eta}_q$$

How does it propagate up to the measurement?

- Position

$$\mathbf{f}_p(\mathbf{q} + \boldsymbol{\eta}_q) \approx \mathbf{f}_p(\mathbf{q}) + \underbrace{\mathbf{J}_c(\mathbf{q})\boldsymbol{\eta}_q}_{\boldsymbol{\eta}_p}$$

- Velocity

$$\mathbf{J}(\mathbf{q} + \boldsymbol{\eta}_q)(\dot{\mathbf{q}} + \boldsymbol{\eta}_{\dot{\mathbf{q}}}) \approx \mathbf{J}(\mathbf{q})\dot{\mathbf{q}} + \underbrace{\frac{\partial}{\partial \mathbf{q}}(\mathbf{J}(\mathbf{q})\dot{\mathbf{q}})\boldsymbol{\eta}_q + \mathbf{J}(\mathbf{q})\boldsymbol{\eta}_{\dot{\mathbf{q}}}}_{\boldsymbol{\eta}_v}$$

# State

Depending on the type of robot and sensors, the state usually is:

- $\mathbf{x}_k = [\mathbf{p}_k, \mathbf{R}_k, \mathbf{v}_k, \mathbf{b}_k^a, \mathbf{b}_k^\omega, \mathbf{c}_1, \mathbf{c}_2, \mathbf{c}_3, \mathbf{c}_4, ]$  for quadrupeds
- $\mathbf{x}_k = [\mathbf{p}_k, \mathbf{R}_k, \mathbf{v}_k, \mathbf{b}_k^a, \mathbf{b}_k^\omega, \mathbf{c}_1, \mathbf{c}_2, \mathbf{B}_1, \mathbf{B}_2, ]$  for humanoids

# System Dynamics

$$\dot{\mathbf{R}} = \mathbf{R}(\tilde{\boldsymbol{\omega}} - \boldsymbol{\eta}_{\omega})^{\wedge}$$

$$\dot{\mathbf{v}} = -\boldsymbol{\omega}^{\wedge} \mathbf{v} + \mathbf{R}^{\top} \mathbf{g}^w + \mathbf{a}$$

$$\dot{\mathbf{p}} = \mathbf{v}$$

$$\dot{\mathbf{b}}_{\omega} = \boldsymbol{\eta}_{\omega}$$

$$\dot{\mathbf{b}}_a = \boldsymbol{\eta}_a$$

# Extended Kalman Filter

- We treat IMU measurements as *inputs*, so the prediction becomes:

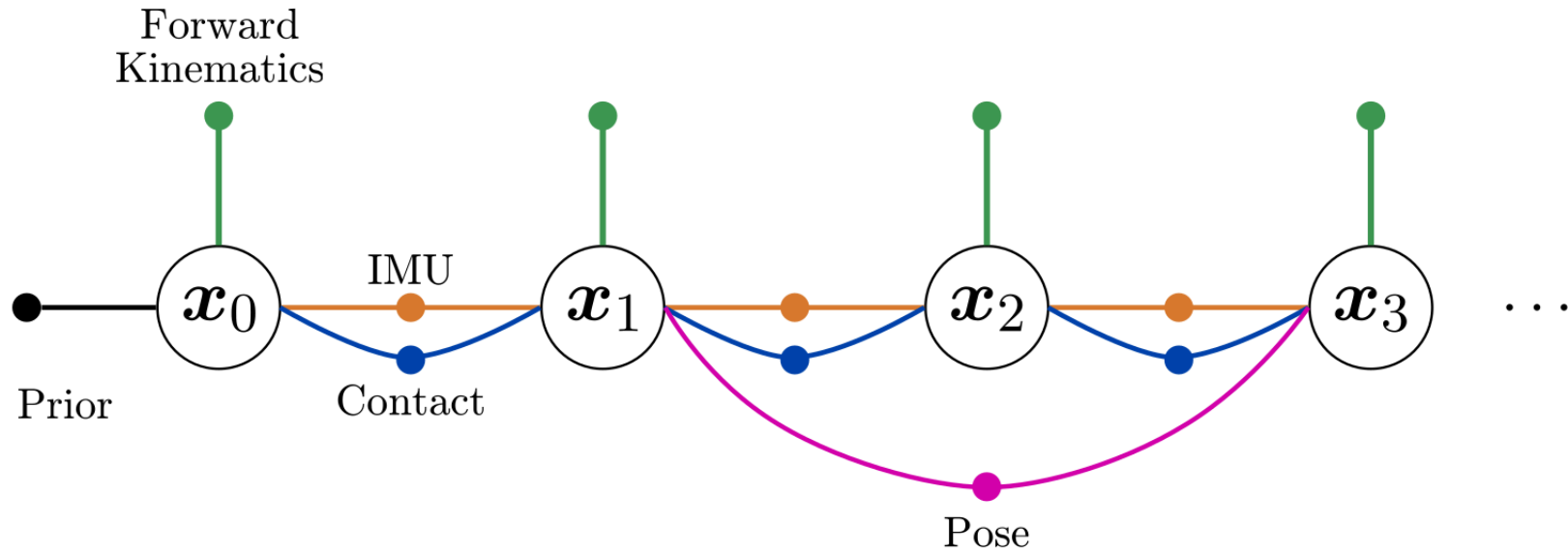
$$\mathbf{u}_k = \begin{bmatrix} \boldsymbol{\omega} \\ \mathbf{a} \end{bmatrix} \quad \boldsymbol{\mu}_k^- = \begin{bmatrix} \mathbf{p}_{k-1} \\ \mathbf{v}_{k-1} \\ \mathbf{0} \\ \mathbf{b}_{k-1}^a \\ \mathbf{b}_{k-1}^\omega \end{bmatrix} + \begin{bmatrix} \mathbf{v}_{k-1} \Delta t \\ (-\boldsymbol{\omega}^\wedge \mathbf{v} + \mathbf{R}^\top \mathbf{g}^w + \mathbf{a}) \Delta t \\ \mathbf{R}_{k-1} \exp(\boldsymbol{\omega}^\wedge \Delta t) \\ \mathbf{0} \\ \mathbf{0} \end{bmatrix}$$

- The covariance  $\Sigma_k^-$  is computed by integration of the process equations
- The measurement is simply  $\mathbf{z}_k = \tilde{\mathbf{v}}$  as per velocity formulation

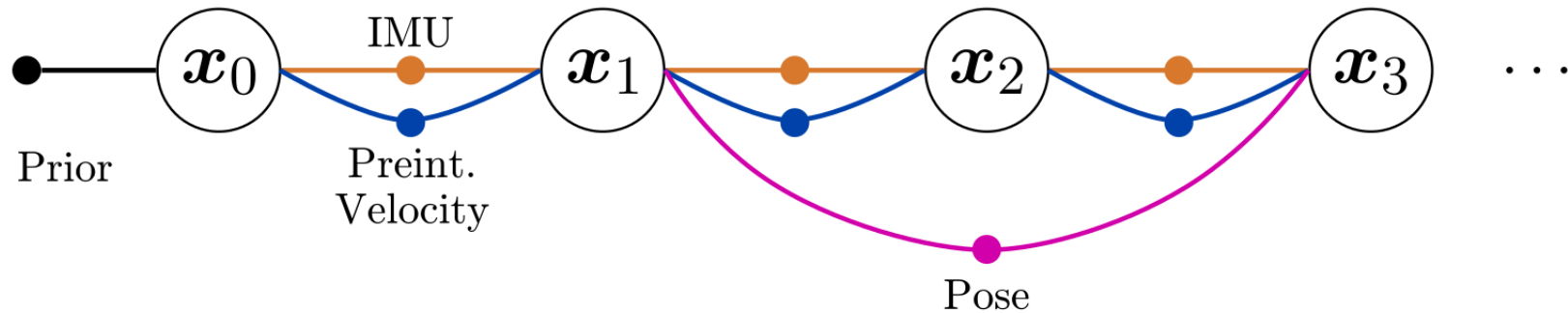
# Smoothing

- The process model and the measurement model need to be expressed as *residuals*
- Normally, residuals connect lower frequency estimates (e.g. Camera rate  $20Hz$ )
- We need to *preintegrate* the measurements between camera frames
- To avoid re-linearization, the preintegrated factors should be independent of the state, they should depend only on the difference between the state the factor connects

# Factor Graphs



(a) *Factor graph with preintegrated contacts*



# IMU Preintegration

IEEE TRANSACTIONS ON ROBOTICS, VOL. 33, NO. 1, FEBRUARY 2017

1

## On-Manifold Preintegration for Real-Time Visual–Inertial Odometry

Christian Forster, Luca Carlone, Frank Dellaert, and Davide Scaramuzza

**Abstract**—Current approaches for visual–inertial odometry (VIO) are able to attain highly accurate state estimation via nonlinear optimization. However, real-time optimization quickly becomes infeasible as the trajectory grows over time; this problem is further emphasized by the fact that inertial measurements come at high rate, hence, leading to the fast growth of the number of variables in the optimization. In this paper, we address this issue by preintegrating inertial measurements between selected keyframes into single relative motion constraints. Our first contribution is a *preintegration theory* that properly addresses the manifold structure of the rotation group. We formally discuss the generative measurement model as well as the nature of the rotation noise and derive the expression for the *maximum a posteriori* state estimator. Our theoretical development enables the computation of all

### I. INTRODUCTION

THE use of cameras and inertial sensors for a three-dimensional (3-D) structure and motion estimation has received considerable attention from the robotics community. Both sensor types are cheap, ubiquitous, and complementary. A single moving camera is an exteroceptive sensor that allows us to measure appearance and geometry of a 3-D scene, up to an unknown metric scale; an inertial measurement unit (IMU) is a proprioceptive sensor that renders metric scale of monocular vision and gravity observable [1] and provides robust and accurate interframe motion estimates. Applications of a visual–

# Contact Preintegration

- Forward kinematics factor

$$\begin{aligned} \mathbf{r}_{\mathcal{F}} &= \text{Log} \left( \mathbf{C}_i^{-1} \mathbf{T} \text{fk}(\tilde{\mathbf{q}}) \right) \\ \Sigma_{\mathcal{F}} &= \mathbf{J}_c(\tilde{\mathbf{q}}) \Sigma_q \mathbf{J}_c^{\top}(\tilde{\mathbf{q}}) \end{aligned}$$

- Contact factor

$$\begin{aligned} \mathbf{r}_{\mathcal{C}} &= \begin{bmatrix} \text{Log} \left( \mathbf{B}_i^{\top} \mathbf{B}_j \right) \\ \mathbf{B}_i^{\top} (\mathbf{c}_j - \mathbf{c}_i) \end{bmatrix} \\ \Sigma_{\mathcal{C}} &= \begin{bmatrix} \Sigma_w & \mathbf{0} \\ \mathbf{0} & \Sigma_v \end{bmatrix} \Delta t_{ij} \end{aligned}$$

# Preintegrated Velocity Factor

$$\Delta \tilde{\mathbf{t}}_{ij} = \Delta \mathbf{t}_{ij} + \delta \mathbf{p}_{ij} = \sum_{k=i}^{j-1} \left[ \Delta \tilde{\mathbf{R}}_{ik} \tilde{\mathbf{v}}_k \Delta t \right] + \delta \mathbf{p}_{ij},$$

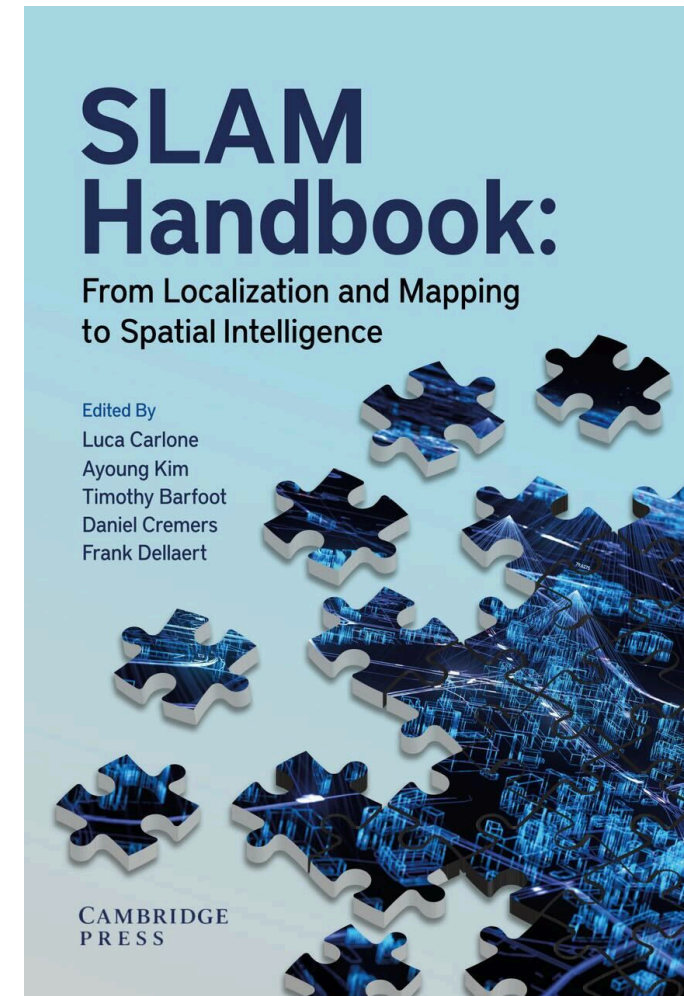
$$\mathbf{r}_{\mathcal{V}} = \mathbf{R}_i^{\top} (\mathbf{t}_j - \mathbf{t}_i) - \Delta \mathbf{t}_{ij}$$

$$\Sigma_{\mathcal{V},ij} = \sum_{k=i}^{j-1} \Sigma_{\mathcal{V},ik} + \mathbf{A} \Sigma_v \mathbf{A}^{\top},$$

with the matrix  $\mathbf{A} = \Delta \tilde{\mathbf{R}}_{ik} \Delta t$ .

# References

- M. Camurri and M. Mattamala  
SLAM Handbook, Chapter 12



# References

- Camurri M, Ramezani M, Nobili S and Fallon M (2020) Pronto: A Multi-Sensor State Estimator for Legged Robots in Real-World Scenarios. *Front. Robot. AI* 7:68. doi: 10.3389/frobt.2020.00068
- D. Wisth, M. Camurri and M. Fallon, "VILENS: Visual, Inertial, Lidar, and Leg Odometry for All-Terrain Legged Robots," in *IEEE Transactions on Robotics*, vol. 39, no. 1, pp. 309-326, Feb. 2023, doi: 10.1109/TRO.2022.3193788.
- R. Hartley, M. G. Jadidi, L. Gan, J. -K. Huang, J. W. Grizzle and R. M. Eustice, "Hybrid Contact Preintegration for Visual-Inertial-Contact State Estimation Using Factor Graphs," 2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Madrid, Spain, 2018, pp. 3783-3790, doi: 10.1109/IROS.2018.8593801.